Ain Shams University
Faculty of Computer & Information Sciences
Scientific Computing Department

Vision-Based Obstacle Detection and Avoidance System for an Autonomous Car Prototype

June 2026

Ain Shams University
Faculty of Computer & Information Sciences
Scientific Computing Department

Vision-Based Obstacle Detection and Avoidance System for an Autonomous Car Prototype

This documentation submitted as required for the degree of bachelors in Computer and Information Sciences.

By

Esraa Bassiouny Mohamed [Scientific Computing Department]
Sara Eslam Mohamed [Scientific Computing Department]
Ziad Yasser Moussa [Scientific Computing Department]
Amr Mohamed Ibrahim [Scientific Computing Department]
Ziad Khaled Ibrahim [Scientific Computing Department]

Under Supervision of

Dr. Manal Tantawi
Associate Professor at Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Aya Nasser
Teaching Assistant, Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

June 2026

Acknowledgements

First and foremost, we would like to express our deepest gratitude to Allah for granting us the strength, patience, determination, and perseverance to successfully complete this project. This journey was filled with challenges, learning experiences, and continuous improvements, and without His guidance, this achievement would not have been possible.

We would like to extend our sincere appreciation and heartfelt thanks to our supervisor, Dr. Manal Tantawi, for her continuous support, valuable guidance, insightful feedback, and encouragement throughout all stages of the project. Her knowledge, experience, and dedication greatly contributed to improving both the technical and research aspects of this work. Her advice always motivated us to push our limits and strive for better results.

We would also like to express our gratitude to our teaching assistant, Eng. Aya Nasser, for her continuous follow-up, technical assistance, constructive suggestions, and support during the implementation, testing, and development phases of the project. Her cooperation and dedication played an important role in helping us overcome many technical challenges.

Special thanks are extended to the Faculty of Computers and Information, Ain Shams University, for providing us with the academic environment, facilities, and learning opportunities that enabled us to develop this project and enhance our technical and practical skills.

Finally, we are deeply thankful to our families and friends for their endless support, encouragement, patience, and motivation throughout this journey. Their belief in us gave us the confidence and determination to continue working hard and achieve our goals successfully.

Abstract

This project presents the development of a Vision-Based Obstacle Detection and Avoidance System for an Autonomous Car Prototype. The system aims to simulate the core functionalities of autonomous vehicles using computer vision, artificial intelligence, embedded systems, and real-time control techniques.

The proposed system utilizes a YOLO-based object detection model trained on a custom merged dataset containing traffic-related objects such as toy cars, traffic lights, and stop signs. The model performs real-time object detection to help the vehicle perceive its surrounding environment accurately and avoid potential obstacles during navigation. In addition, lane detection algorithms are implemented to ensure stable lane following and intelligent steering control by continuously adjusting the vehicle direction based on lane position.

The system is deployed on a Raspberry Pi (RPI) 5, which acts as the main processing unit responsible for image processing, decision-making, obstacle avoidance, and steering control. An Arduino microcontroller is used for low-level motor control and vehicle movement execution through PWM-based speed regulation.

To improve reliability and safety, the system integrates ultrasonic sensors with the vision-based model using sensor fusion techniques. This enhances obstacle distance estimation, improves collision avoidance, and reduces false detections during operation. The project demonstrates the integration between artificial intelligence, robotics, embedded systems, computer vision, and control engineering in a practical real-time application.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
CNN	Convolutional Neural Network
CV	Computer Vision
DL	Deep Learning
DQN	Deep Q-Network
FPS	Frames Per Second
GPIO	General Purpose Input/Output
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	Mean Average Precision
PWM	Pulse Width Modulation
ROI	Region of Interest
UART	Universal Asynchronous Receiver-Transmitter
YOLO	You Only Look Once
BEV	Bird's Eye View
HSV	Hue, Saturation, Value
BGR	Blue, Green, Red
RGB	Red, Green, Blue
NMS	Non Maximum Suppression
OpenCV	Open Source Computer Vision Library
ONNX	Open Neural Network Exchange
RPI	Raspberry Pi
AVs	Autonomous vehicles

Chapter One
Introduction

Chapter 1: Introduction

In recent years, autonomous driving systems have become one of the most important research areas in artificial intelligence, robotics, and intelligent transportation systems. The rapid growth in road accidents caused by human errors such as distraction, fatigue, delayed reaction time, and poor decision-making has increased the demand for safer and smarter transportation technologies. Autonomous vehicles aim to minimize these risks by enabling vehicles to perceive their surroundings, analyze road conditions, and make driving decisions automatically without continuous human intervention.

Computer vision and deep learning technologies have played a major role in advancing autonomous driving systems. Modern object detection algorithms, especially YOLO (You Only Look Once), provide high-speed and accurate real-time detection capabilities that are suitable for autonomous navigation tasks. These models allow vehicles to recognize obstacles, vehicles, traffic signs, and lane boundaries efficiently while operating in dynamic environments.

This project presents the design and implementation of a Vision-Based Obstacle Detection and Avoidance System for an Autonomous Car Prototype. The proposed system focuses on developing a small-scale autonomous vehicle capable of detecting surrounding obstacles, recognizing traffic-related objects, following lanes, and avoiding collisions intelligently in real time.

1.1 Problem Definition

Conventional driving relies entirely on human perception, which is prone to errors including delayed reaction, distraction, and traffic rule violations. These factors contribute to approximately 1.19 million deaths annually due to road accidents, with over 90% caused by human error. Existing autonomous systems are often complex and expensive, creating a strong need for a low-cost, vision-based intelligent system.

The core problem addressed in this project is designing a real-time autonomous vehicle prototype capable of:

- Detecting and classifying road obstacles accurately using computer vision.
- Avoiding collisions dynamically in real time.
- Following lanes autonomously using classical image processing.
- Recognizing traffic signs and traffic lights with deep learning.
- Making intelligent driving decisions based on sensor inputs.
- Integrating computer vision with sensor-based perception for reliable operation.

1.2 Motivation

The motivation behind this project arises from the increasing need for safer, smarter, and more reliable transportation systems. The following points summarize the main motivations:

- Human error is responsible for the majority of road accidents worldwide, creating a strong need for intelligent driver-assistance systems.
- Vision-based autonomous systems provide a low-cost alternative to expensive sensor-heavy autonomous vehicle solutions such as LiDAR-based systems.
- Deep learning models such as YOLO offer powerful real-time object detection capabilities suitable for embedded autonomous systems.
- Combining computer vision with sensor fusion improves obstacle detection reliability and reduces false detections.
- Developing a small-scale autonomous prototype allows practical experimentation with real-world autonomous driving concepts in an educational environment.
- The project provides valuable hands-on experience in artificial intelligence, robotics, embedded systems, and control engineering.

1.3 Objectives

The main goal is to develop a self-driving car prototype with real-time perception capabilities. The key objectives are:

- Develop a real-time object detection system using YOLOv8 trained on a custom merged dataset.
- Implement lane detection and steering control algorithms for autonomous navigation.
- Design an obstacle avoidance system capable of detecting and reacting to obstacles dynamically.
- Integrate ultrasonic sensors with vision-based perception (sensor fusion) to improve environmental awareness.
- Build a real hardware prototype using RPI 5, Arduino, camera, and motor driver modules.
- Combine AI, sensors, and control modules into a unified autonomous driving system.

1.4 Time Plan

The project was planned over two semesters following the Gantt chart structure shown in Table 1.1.

Table 1.1: Project Time Plan

Task	Phase 1	Phase 2	Phase 3	Phase 4	Phase 5
Literature Review & Survey	✓				
Dataset Collection & Preprocessing	✓				
YOLO Model Training & Testing	✓		✓		✓
Lane Detection & Hardware Integration	✓			✓	
Testing, Results & Documentation	✓				✓

1.5 Documentation Outline

This documentation is organized as follows:

- Chapter 1 – Introduction: Presents the problem definition, motivation, objectives, and project scope.
- Chapter 2 – Background: Provides a literature review covering autonomous vehicles, object detection, YOLO, lane detection, and sensor fusion techniques.
- Chapter 3 – System Architecture: Describes the overall system design and the main components and their interactions.
- Chapter 4 – System Implementation: Details the dataset preparation, YOLO model training, lane detection algorithm, hardware implementation, and integration.
- Chapter 5 – System Testing: Documents the testing methodology, test cases, and results with screenshots.
- Chapter 6 – Conclusion and Future Work: Summarizes project achievements and identifies areas for future enhancement.

Chapter 2
Background

Chapter 2: Background

This chapter provides the necessary literature review, survey of existing work, and technical background required to understand the technologies and methods used in this project.

2.1 Autonomous Vehicles Overview

Autonomous vehicles (AVs) are self-driving systems that can navigate and operate without direct human input. Levels of autonomy range from Level 0 (no automation) to Level 5 (full automation). Achieving higher autonomy levels requires the integration of multiple technologies including computer vision, sensor fusion, machine learning, and real-time control. Early research by Bojarski et al. [1] demonstrated end-to-end deep learning for self-driving cars, establishing the foundation for modern approaches. Building on this direction, Chen et al. [2] proposed a direct-perception affordance approach that maps camera input to a compact set of driving indicators.

2.2 Object Detection Techniques

Object detection algorithms can be broadly categorized into:

- Traditional Computer Vision: Edge detection, Hough Transform, and template matching are used in low-cost applications where computational resources are limited.
- Deep Learning-Based Detection: Convolutional Neural Networks (CNNs) and models such as Faster R-CNN, SSD, and YOLO provide significantly higher accuracy and speed for complex real-world environments.

Huang et al. [3] analyzed speed/accuracy trade-offs for modern convolutional object detectors, providing important guidance for selecting detection architectures for real-time applications.

2.3 YOLO Object Detection Models

YOLO (You Only Look Once), introduced by Redmon et al. [4], revolutionized real-time object detection by processing the entire image in a single forward pass of the neural network. Key advantages include:

- Real-time detection speed suitable for embedded systems.
- High accuracy in detecting multiple object classes simultaneously.
- Effective detection of both large and small objects.
- Compatibility with custom-trained datasets.

YOLOv8, the latest iteration at the time of this project and maintained by Ultralytics [5], provides improved accuracy with a streamlined architecture. XNOR-Net concepts [6] also inspired binary neural network optimizations for embedded environments.

2.4 Lane Detection Techniques

Lane detection is a critical component for autonomous navigation. Techniques include:

- Classical CV: Canny edge detection, Gaussian blur, binary thresholding, and Hough Transform for line segment detection.
- Deep Learning-Based: CNN-based semantic segmentation approaches, though more computationally expensive.

For this project, classical computer vision techniques (OpenCV) were chosen for lane detection due to their efficiency on the RPI 5 embedded platform.

2.5 Obstacle Avoidance Systems

Obstacle avoidance systems enable a vehicle to detect and react to obstacles in its path. Approaches include:

- Rule-Based Systems: Predefined logic for braking, slowing, and steering based on obstacle position.
- Reinforcement Learning: DQN and other RL approaches for advanced adaptive systems.
- Sensor-Based: Ultrasonic sensors for proximity detection combined with vision for classification.

Lee and Peng [7] evaluated automotive forward collision warning and avoidance algorithms, demonstrating the importance of rapid decision-making in collision avoidance contexts.

2.6 Sensor Fusion

Sensor fusion combines data from multiple sensors to achieve more reliable environmental perception than any single sensor alone. In this project, ultrasonic sensors are fused with the camera-based YOLO model to improve obstacle distance estimation and reduce false detections. Stiller et al. [8] demonstrated the effectiveness of multisensor obstacle detection and tracking in autonomous systems.

Limitation of existing work: Most existing systems focus on a single task rather than a fully integrated autonomous system. This project addresses that gap by combining object detection, lane detection, obstacle avoidance, and hardware control into one unified prototype.

Chapter 3
System Architecture

Chapter 3: System Architecture

The system architecture describes the main components of the autonomous car prototype and the interactions between these components. The system is composed of five main modules: Vision System, Processing Unit, Control Unit, Mobility System, and Power Supply.

3.1 Overall System Design

The system follows a pipeline architecture where data flows from image capture through processing to motor actuation:

- Camera captures real-time video frames from the front of the vehicle.
- Frames are transmitted to the RPI 5 for processing.
- The RPI runs YOLO object detection and lane detection algorithms in parallel.
- Decisions are made based on detected objects and lane position.
- Control signals are sent via serial communication to the Arduino.
- The Arduino drives the motors through the L298N motor driver.

Figure 3.1 shows the overall system architecture, illustrating how the main components of the autonomous car prototype are connected and interact with one another.

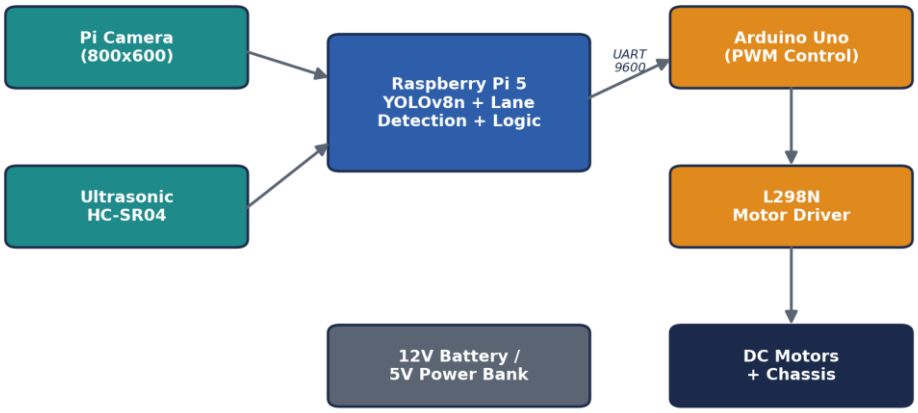


Figure 3.1: System Architecture Block Diagram

3.2 Hardware Architecture

The hardware components of the prototype and their roles are summarized in Table 3.1.

Table 3.1: Hardware Components and Specifications

Component	Specification / Role
RPI 5 (8GB)	Main processing unit — runs YOLO + lane detection + decision logic
Arduino Uno R3	Hardware control unit — executes motor commands via PWM
Camera Module	Front-facing camera for real-time video frame capture
Ultrasonic Sensor	Distance measurement for obstacle proximity detection
L298N Motor Driver	Powers and controls DC motors from Arduino signals
DC Motors	Provide wheel drive via chassis and gearbox
12V Battery	Primary power supply for motors and motor driver
5V Power Bank	Powers RPI 5 (5V / 5A)
Chassis & Wheels	Mechanical platform supporting all hardware components

3.3 Software Architecture

The main software modules of the system and the technologies used to implement them are summarized in Table 3.2.

Table 3.2: Software Modules and Functions

Module	Technology / Function
Object Detection	YOLOv8 trained on custom merged dataset
Lane Detection	OpenCV — Gaussian blur, Hough Transform, P-control
Decision Making	Rule-based logic for stop, slow down, and steer commands
Serial Communication	UART serial link between RPI and Arduino
Motor Control	PWM-based speed regulation via Arduino

3.4 Data Flow

Camera → [Frame Capture] → RPI 5 → [YOLO Detection + Lane Detection] → [Decision & Control Logic] → [Serial Command] → Arduino Uno → [Motor Driver] → DC Motors

Figure 3.2 shows the end-to-end data flow pipeline of the system, tracing each processing stage from frame capture through to motor actuation.

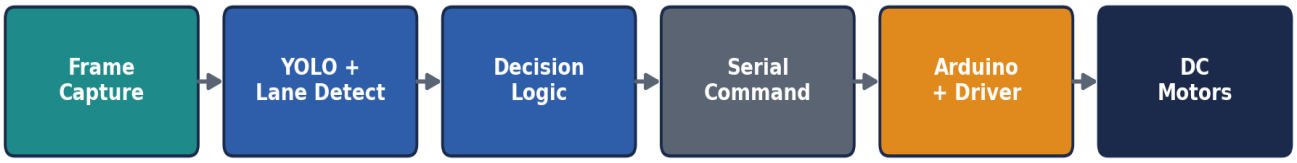


Figure 3.2: End-to-End Data Flow Pipeline

Chapter 4
System Implementation

Chapter 4: System Implementation

4.1 Dataset Preparation and Preprocessing

Since the datasets were collected from different sources and had different class structures, a dataset merging pipeline was developed to unify all annotations into a single consistent format.

During the merging process:

- Images and labels from all datasets were combined into one directory structure.
- Class IDs were remapped into unified class indices.
- Duplicate naming conflicts were avoided using automatic file renaming.
- Training, validation, and testing splits were preserved.
- All labels were converted into YOLO annotation format.

This unified dataset allowed the model to learn multiple driving-related objects simultaneously within a single training pipeline.

4.1.1 Image Preprocessing

Several preprocessing techniques were applied before training to improve data consistency and training stability.

1. Image Resizing

All images were resized to 320 × 320 pixels using letterbox resizing, which scales each image while preserving its original aspect ratio and pads the remaining area with a neutral gray border. This matches the input resolution used during YOLO training and avoids distorting object shapes. Resizing provided several advantages:

- Reduced computational cost on RPI hardware.
- Faster inference speed.
- Consistent input dimensions for neural network processing.
- Improved real-time performance.

2. Data Normalization

Pixel values were normalized from the original range [0, 255] to [0, 1] using standard min-max normalization. Normalization improves neural network convergence by ensuring all input values remain within a consistent numerical range. It also reduces training instability caused by large pixel variations.

$$Normalized\ Pixel = \frac{Pixel\ Value}{255}$$

3. Label Verification

Bounding box annotations were validated to ensure:

- Correct object coordinates.
- Valid class mappings.
- No corrupted labels.
- Proper YOLO formatting.

This step was necessary because incorrect annotations can significantly reduce object detection accuracy.

4.1.2 Data Augmentation

To improve model generalization and reduce overfitting, several augmentation techniques were applied during preprocessing. Data augmentation artificially increases dataset diversity by generating modified versions of existing images while preserving object labels. Augmentation was targeted at the under-represented (rare) classes — Lego person, stop sign, and the green, red, and yellow traffic lights — generating roughly three augmented copies per rare-class image to improve class balance. In total, 4,599 augmented images were added, expanding the training set from 3,242 to 7,311 samples.

Horizontal Flip

Images were randomly flipped horizontally to simulate vehicles and obstacles appearing from different directions. This improves directional robustness, prevents left/right positional bias, and enhances lane-side generalization.

Scaling Transformations

Random scaling operations were applied to simulate objects appearing at different distances from the camera. This helps detect near and far objects, improves perspective adaptation, and enhances detection under varying distances.

Saturation and Brightness Adjustments

Lighting and color intensity were randomly modified to simulate different environmental conditions. This improves robustness to illumination changes, simulates day/night brightness variation, and reduces sensitivity to camera exposure.

Motion Blur Simulation

Motion blur effects were applied to mimic vehicle movement and camera vibration during driving. This improves robustness during motion, simulates real driving conditions, and enhances real-time detection stability.

Figure 4.1 shows sample images from the final preprocessed dataset after the augmentation techniques described above were applied.

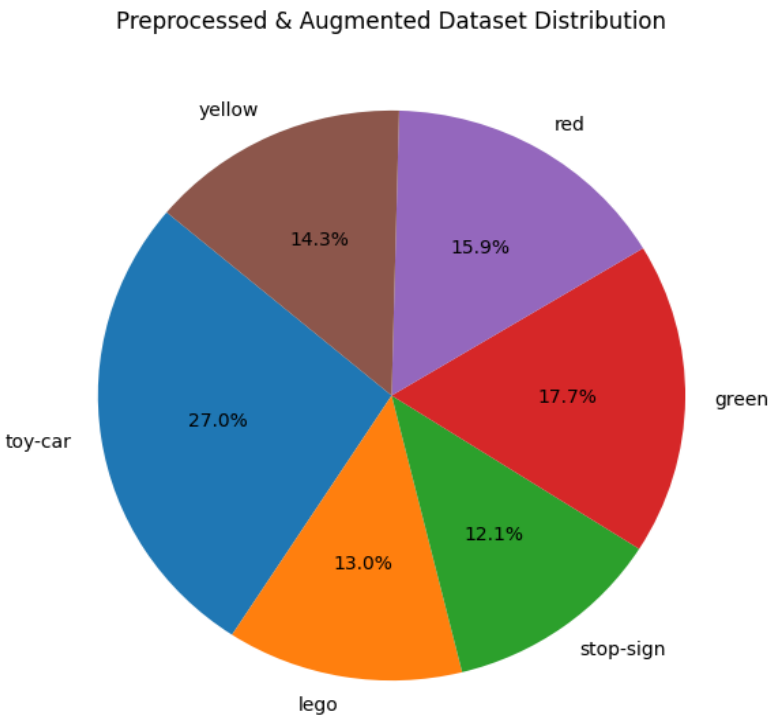


Figure 4.1: Sample Images from the Final Preprocessed Dataset

4.1.3 Final Preprocessed Dataset

After preprocessing and augmentation, the dataset size increased from 3,242 merged images to 7,311 training samples. The final dataset provided better class balance, increased environmental diversity, improved generalization capability, and higher robustness during real-time deployment on Raspberry Pi.

Table 4.1 summarizes the dataset statistics before and after the preprocessing and augmentation steps.

Table 4.1: Dataset Statistics Before and After Preprocessing

Dataset Metric	Value
Total Merged Images	3,242
After Augmentation / Preprocessing	7,311 training samples (4,599 augmented images added to rare classes)
Image Resolution	320 × 320 pixels
Normalization Range	[0, 1]
Per-Class Instances (after aug.)	toy-car 2,822; lego 1,361; stop-sign 1,266; green 1,851; red 1,669; yellow 1,496
Classes	6 (Toy Car, Lego Person, Stop Sign, Green/Red/Yellow Light)

This preprocessing pipeline significantly improved the final YOLO model performance and helped achieve stable real-time detection under different driving conditions.

Figure 4.2 shows the class distribution of the final merged dataset across the six object categories.

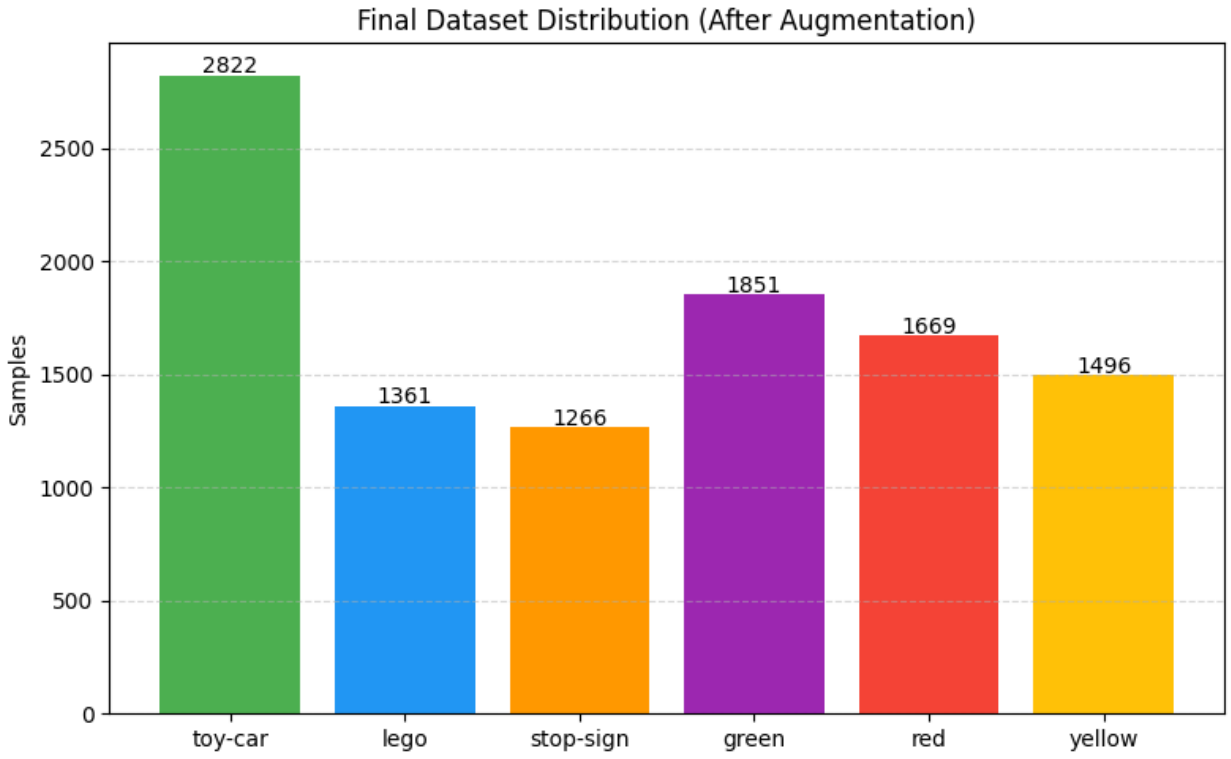


Figure 4.2: Final Dataset Class Distribution

4.2 YOLO Object Detection Module

Object detection is one of the most important components in autonomous driving systems because the vehicle must continuously perceive and understand its surrounding environment in real time. The detection system must identify multiple road objects simultaneously, estimate their positions accurately, and provide fast responses for safe decision-making.

In this project, the YOLOv8n (You Only Look Once Version 8 Nano) model was selected as the primary object detection algorithm due to its excellent balance between detection accuracy, real-time inference speed, lightweight architecture, low computational requirements, and compatibility with embedded systems such as Raspberry Pi.

The trained YOLO model is responsible for detecting toy vehicles, Lego pedestrians, stop signs, and traffic lights (red, yellow, green). These detections are then used by the autonomous driving logic to perform actions such as obstacle avoidance, steering correction, speed adjustment, stopping at traffic signs, traffic light response, and auto-parking assistance.

4.2.1 Why YOLOv8n Was Selected

Several object detection models were studied during the research phase, including Faster R-CNN, SSD (Single Shot Detector), YOLOv5, and YOLOv8. Although two-stage detectors such as Faster R-CNN provide high accuracy, they require significant computational resources and are relatively slow for embedded real-time systems.

The project required a model capable of running on RPI hardware, performing real-time inference, detecting multiple objects simultaneously, maintaining acceptable accuracy, and operating with limited GPU/CPU resources. For these reasons, YOLOv8n was selected.

4.2.2 Advantages of YOLOv8n

1. Real-Time Performance

YOLO processes the entire image in a single forward pass through the neural network, unlike traditional two-stage detectors that first generate regions and then classify them. This architecture enables real-time camera processing, continuous object tracking, immediate decision making, and smooth autonomous driving behavior. This is critical for autonomous systems where delayed decisions may lead to collisions.

2. Lightweight Architecture

The YOLOv8n (Nano) version is specifically optimized for edge devices and embedded systems with a smaller model size, lower memory usage, faster inference time, reduced CPU/GPU load, and lower power consumption. Compared to larger YOLO variants, YOLOv8n sacrifices a small amount of accuracy in exchange for significantly better speed and efficiency on RPI hardware.

3. Improved Detection Accuracy

YOLOv8 introduced several architectural improvements over earlier YOLO versions, including anchor-free detection, better feature pyramid networks, improved backbone structure, enhanced small-object detection, and better localization accuracy. These improvements were particularly useful since the system must detect small toy cars, small traffic lights, tiny stop signs, and Lego figures under varying lighting conditions and camera angles.

4. Multi-Class Detection Capability

The model was trained to detect multiple classes simultaneously using a unified dataset. YOLOv8 efficiently handles multiple objects in one frame, overlapping objects, different object scales, and real-time multi-class classification. This allows the autonomous vehicle to understand complex driving scenes using a single neural network.

5. Easy Integration with Raspberry Pi

YOLOv8 provides excellent Python support through the Ultralytics framework, which simplified deployment on Raspberry Pi. Benefits include a simple inference pipeline,

Open Neural Network Exchange (ONNX) export support, easy OpenCV integration, fast model loading, and real-time camera processing compatibility. The trained model was exported and integrated directly into the Raspberry Pi-based driving system.

4.2.3 YOLO Training Pipeline

The YOLOv8n model was trained using the merged and preprocessed custom dataset. The model learned to classify and localize six object categories, as listed in Table 4.2:

Table 4.2: YOLO Class ID Mapping

Class ID	Object
0	Toy Car
1	Lego Person
2	Stop Sign
3	Green Light
4	Red Light
5	Yellow Light

4.2.4 Training Configuration

The training process used the Ultralytics YOLOv8 framework [5], which builds on the single-pass detection paradigm originally introduced by Redmon et al. [4], with custom hyperparameters optimized for embedded deployment, as summarized in Table 4.3:

Table 4.3: YOLOv8n Training Configuration

Parameter	Value
Model	YOLOv8n (trained from scratch)
Input Resolution	320 × 320
Batch Size	32
Epochs	150 (early-stopped at 91)
Framework	Ultralytics YOLO
Optimizer	SGD (momentum 0.937, weight decay 0.0005)
Initial Learning Rate	0.01 (final LR factor 0.01)
Early Stopping Patience	20 epochs
Training Device	Kaggle — Tesla T4 GPU
Deployment Device	RPI 5 (8GB)

4.2.5 YOLO Model Performance Results

The trained model achieved strong detection performance across all object categories, as summarized in Table 4.4:

Table 4.4: Overall Object Detection Performance Metrics

Metric	Value
Precision	96.8%
Recall	95.6%
mAP@50	98.0%
mAP@50-95	79.7%

Figure 4.3 shows the training loss curves of the YOLOv8n model over the training epochs, including the box, classification, and DFL loss components.

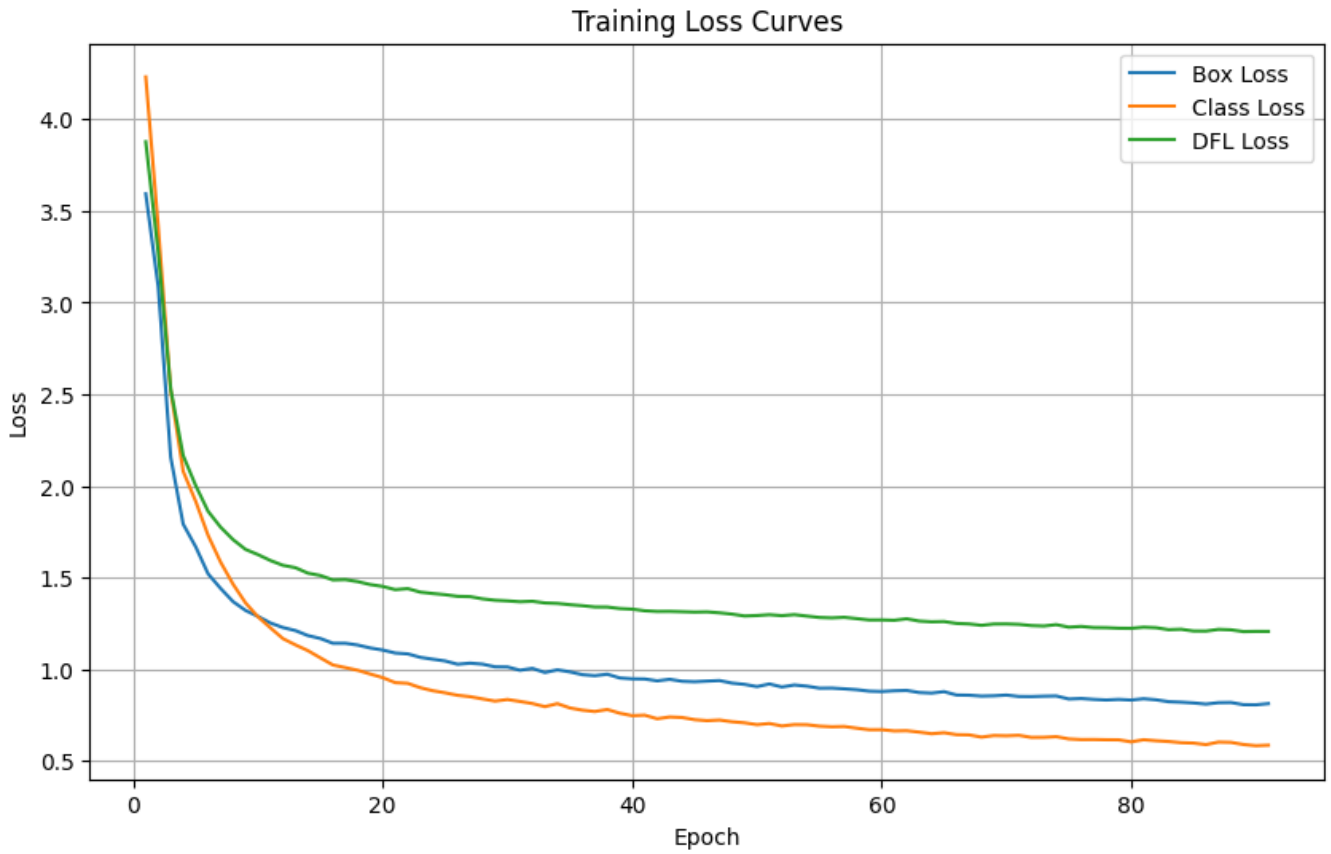


Figure 4.3: YOLOv8n Training Loss Curves

The per-class detection performance is presented in Table 4.5.

Object Class	mAP@50-95
Toy Car	0.850
Lego Person	0.826
Stop Sign	0.889
Green Light	0.685
Red Light	0.728
Yellow Light	0.805

Table 4.5: Per-Class Detection Performance (mAP@50-95)

4.2.6 Performance Analysis

High Precision (96.8%)

Most detected objects were classified correctly with very few false positives. This reduces incorrect vehicle decisions, improves obstacle avoidance reliability, and prevents false traffic sign responses.

High Recall (95.6%)

The model successfully detected most objects present in the scene, resulting in a lower probability of missing obstacles, better driving safety, and more reliable autonomous behavior.

Strong Stop Sign Detection

The stop sign class achieved the highest detection performance because the object shape is highly distinctive, color contrast is strong, and dataset quality was consistent. This improved traffic-rule compliance reliability.

Traffic Light Detection Challenges

Traffic light classes achieved slightly lower performance due to small object size, lighting sensitivity, motion blur, and similar color distributions. Despite these challenges, the model maintained stable real-time detection performance suitable for the prototype environment.

4.2.7 Real-Time System Integration

The trained YOLOv8n model was integrated into the RPI autonomous driving pipeline using OpenCV, Picamera2, the Ultralytics inference engine, and ONNX optimized deployment.

The system performs real-time camera capture, YOLO object detection, bounding box extraction, object classification, decision making, and steering and speed control. Detected objects are then passed to the driving controller to trigger autonomous actions such as:

- Stop at stop signs.
- Slow down near obstacles.
- Avoid collisions.
- Adjust steering direction.
- Validate parking slots.

4.3 Lane Detection Module

The lane detection module is a fundamental component of the autonomous driving system, responsible for keeping the vehicle centered within the driving lane and generating steering commands in real time. The module was implemented in Python using the OpenCV library and deployed on a RPI 5 connected to a RPI Camera (Picamera2). Unlike deep learning-based lane detection approaches, this module relies on classical computer vision techniques, making it lightweight, computationally efficient, and suitable for real-time embedded deployment.

The overall lane-following pipeline consists of the following stages:

- Image Acquisition
- Perspective Transformation
- Lane Color Segmentation
- Sliding Window Lane Extraction
- Polynomial Lane Fitting
- Lane Center Estimation
- Steering Error Calculation
- PWM Motor Control
- Serial Communication with Arduino
- Safety and Fail-Safe Mechanisms

4.3.1 Image Acquisition

The RPI Camera continuously captures Red, Green, Blue (RGB) frames at a resolution of:

$$800 \times 600$$

using the Picamera2 framework.

Each captured frame is converted into OpenCV Blue, Green, Red (BGR) format and forwarded to the lane detection pipeline for processing.

The system processes frames continuously in real time, enabling autonomous lane-following behavior while maintaining low computational overhead.

4.3.2 Perspective Transformation (Bird's-Eye View)

One of the most important preprocessing stages is the perspective transformation.

Road lanes captured by a front-facing camera appear converged due to perspective distortion. This makes accurate lane estimation difficult. To overcome this issue, a bird’s-eye-view (BEV) transformation is applied.

The transformation maps the original camera image into a top-down view using a homography matrix:

M=cv2.getPerspectiveTransform(src,dst)

where:

src = four manually selected points surrounding the lane region.
dst = destination rectangle coordinates.

The transformed image is computed using:

$$I_{warped} = M \times I$$

Advantages:

- Removes perspective distortion.
- Makes lane lines approximately parallel.
- Simplifies lane geometry estimation.
- Improves steering accuracy.

A Region of Interest (ROI) is also visualized during runtime to assist calibration and debugging.

4.3.3 Lane Color Segmentation

After perspective transformation, lane markings must be isolated from the road surface.

Since the track contains white lane boundaries, the image is converted from BGR to Hue, Saturation, Value (HSV) color space:

HSV=f(BGR)

HSV provides better robustness to illumination changes compared to RGB.

White lane pixels are extracted using thresholding:

lower_white = [0,0,220]
upper_white = [180,10,255]

The resulting binary mask contains:

White pixels → lane markings
Black pixels → background

This stage significantly reduces image complexity and improves subsequent lane extraction accuracy.
The corresponding implementation converts each warped frame to the HSV colour space and thresholds for white pixels, producing the binary mask shown below.

```
def threshold_white(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    lower_white = np.array([0, 0, 220])
    upper_white = np.array([180, 10, 255])
    return cv2.inRange(hsv, lower_white, upper_white)
```

Listing 4.1: HSV white-lane segmentation (lane_following.py)

Figure 4.4 shows the warped BEV binary mask, in which the two white lane boundaries appear as near-parallel lines after the perspective transformation.

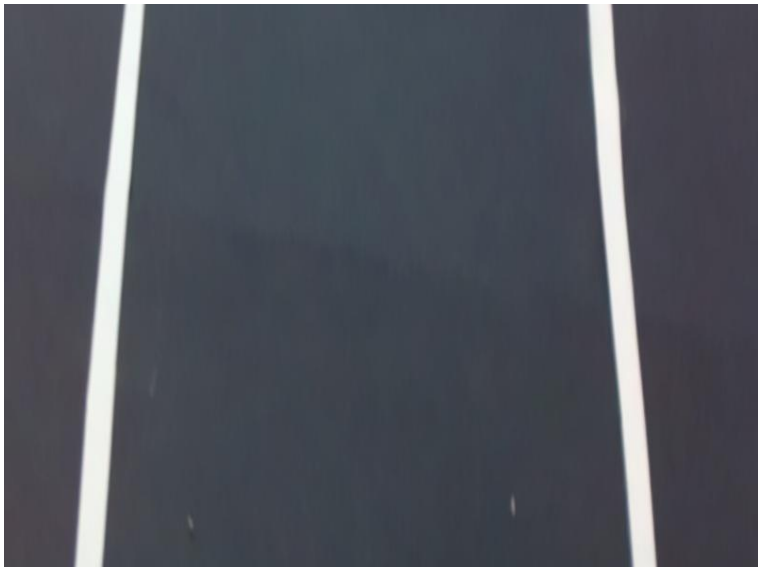


Figure 4.4: Warped BEV

Figure 4.5 shows the resulting binary mask, in which the two white lane boundaries are isolated from the background.

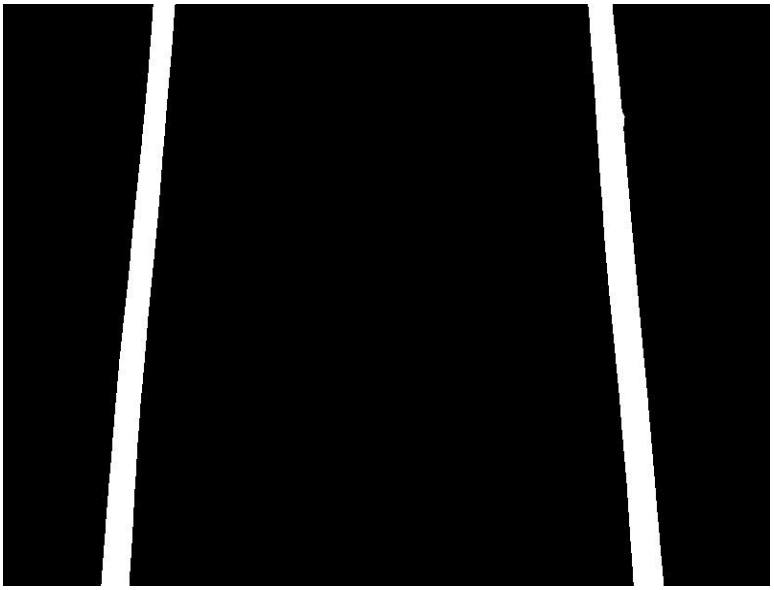


Figure 4.5: Binary Mask Isolating the Two White Lane Boundaries

4.3.4 Histogram-Based Lane Localization

To identify the approximate location of the lane boundaries, a histogram is computed from the lower portion of the binary image.

The histogram is calculated as:

$$H(x) = \sum_{y=[0.72*h]}^{h-1} Binary(y, x)$$

where:

x represents image columns.
High values indicate large concentrations of white pixels.

The histogram typically exhibits two peaks:

Left lane boundary
Right lane boundary

These peaks provide the initial positions for the sliding window search algorithm.

Advantages:

- Fast computation.
- Robust initialization.
- Suitable for real-time deployment.

4.3.5 Sliding Window Lane Detection

After locating the histogram peaks, a sliding window algorithm is used to track lane pixels.

The image is divided into:

$$N=9$$

horizontal windows.

Each window searches for white pixels within a predefined margin:

$$margin = 60$$

If sufficient lane pixels are found:

$$minpix = 50$$

the window center is recentered according to the mean position of the detected pixels.

The process continues from the bottom of the image toward the top, progressively following both lane boundaries.

Advantages:

Robust against partial occlusion.
Handles curved lanes.
Works even when lane markings are partially missing.

Detected lane pixels are visualized for debugging:

Blue → Left lane
Red → Right lane
This stage is implemented as the sliding_window function, which computes the histogram peaks and iteratively recentres each window on the detected lane pixels, as shown in the listing and debug output below.

```
def sliding_window(binary_warped):
    histogram = np.sum(binary_warped[int(binary_warped.shape[0]*0.72):, :], axis=0)
    midpoint = histogram.shape[0] // 2
    leftx_current  = np.argmax(histogram[:midpoint])
    rightx_current = np.argmax(histogram[midpoint:]) + midpoint

    nwindows, margin, minpix = 9, 60, 50
    window_height = binary_warped.shape[0] // nwindows
    nonzero = binary_warped.nonzero()
    nonzeroy, nonzerox = np.array(nonzero[0]), np.array(nonzero[1])

    for window in range(nwindows):
        win_y_low  = binary_warped.shape[0] - (window+1)*window_height
        win_y_high = binary_warped.shape[0] - window*window_height
        # ... select white pixels inside each window (margin) ...
        if len(good_left)  > minpix: leftx_current  = int(np.mean(nonzerox[good_left]))
        if len(good_right) > minpix: rightx_current = int(np.mean(nonzerox[good_right]))
    # returns left/right lane pixel coordinates
```

Listing 4.2: Sliding-window lane tracking (lane_following.py)

Figure 4.6 shows the sliding-window debug view, where the detected left and right lane pixels are highlighted inside the green search windows.

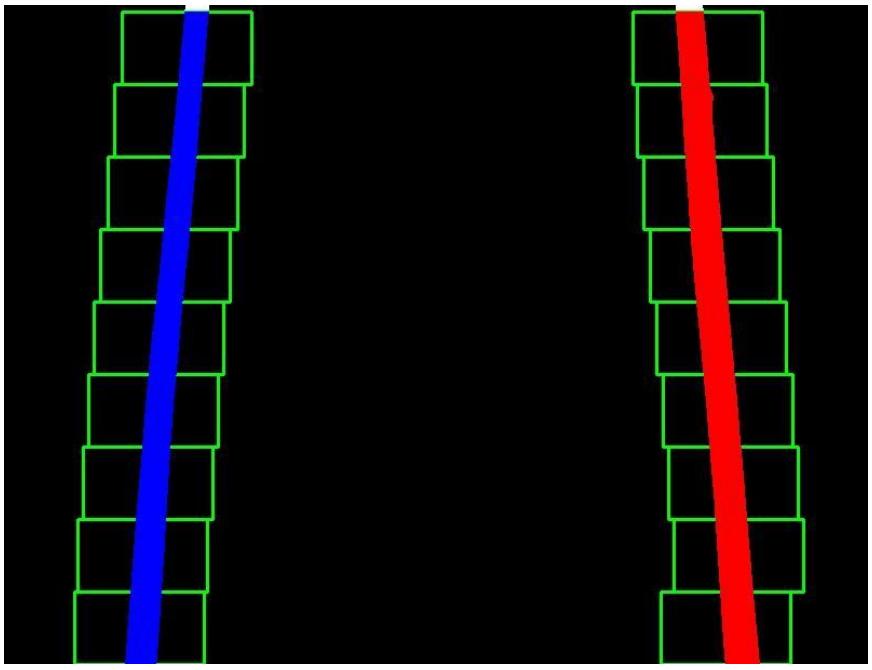


Figure 4.6: Sliding Window Debug View

4.3.6 Polynomial Curve Fitting

Once lane pixels are extracted, a second-order polynomial is fitted to each lane boundary.

The lane model is represented as:

$$x = Ay^2 + By + C$$

using:

np.polyfit(y, x, 2)

for both left and right lanes.

The polynomial representation enables:

Smooth lane estimation.
Curvature handling.
Noise reduction.

Special handling is included for cases where only one lane boundary is visible.

When only one lane boundary is detected, the missing boundary is estimated using a fixed lane

width (580 pixels) determined during system calibration.

This ensures continuous operation even under partial lane loss conditions.
In code, a second-order polynomial is fitted to each boundary using np.polyfit, with a fallback that reconstructs a missing line from the expected lane width of 580 pixels.

```
def fit_polynomial(binary_warped):
    leftx, lefty, rightx, righty = sliding_window(binary_warped)
    left_valid, right_valid = len(leftx) >= 50, len(rightx) >= 50
    lane_width = 580  # expected pixel distance between the two lines

    if left_valid and right_valid:
        left_fit  = np.polyfit(lefty,  leftx,  2)
        right_fit = np.polyfit(righty, rightx, 2)
    elif left_valid:                    # reconstruct missing right line
        left_fit = np.polyfit(lefty, leftx, 2)
        right_fit = left_fit.copy(); right_fit[2] += lane_width
    elif right_valid:                   # reconstruct missing left line
        right_fit = np.polyfit(righty, rightx, 2)
        left_fit = right_fit.copy(); left_fit[2] -= lane_width
    else:
        return None, None
    return left_fit, right_fit
```

Listing 4.3: Polynomial fitting with single-line fallback (lane_following.py

4.3.7 Lane Center Estimation

After fitting both lane boundaries, the center of the lane is computed.

For a given image row:

$$LaneCenter = \frac{LeftLane + RightLane}{2}$$

The vehicle center is defined as:

$$CarCenter = \frac{ImageWidth}{2}$$

The lane center represents the desired trajectory, while the vehicle center represents the current position.

4.3.8 Steering Error Calculation

The steering error is calculated as the horizontal deviation between the lane center and the vehicle center:

Error=LaneCenter−CarCenter

Interpretation:

Positive Error → Vehicle is left of lane center → steer right.
Negative Error → Vehicle is right of lane center → steer left.
Zero Error → Vehicle centered.

To reduce oscillations and measurement noise, temporal smoothing is applied:

$$Error_{smooth} = 0.3 \times Error_{previous} + 0.7 \times Error_{current}$$

This produces more stable steering behavior.

4.3.9 Steering and Motor Control

The steering error is converted into differential motor speeds.

The system uses a proportional steering controller:

$$Adjust = \frac{Error}{80} \times MaxAdjust$$

$$MaxAdjust = 130$$

Motor PWM values are calculated as:

LeftPWM=BaseSpeed+Adjust
RightPWM=BaseSpeed−Adjust

where:
BaseSpeed = 135

PWM values are constrained within:

$$0 \leq PWN \leq 255$$

Interpretation:
Turn Right → Left motor faster.
Turn Left → Right motor faster.
Straight → Equal motor speeds.

This differential-drive mechanism enables smooth lane tracking without requiring a dedicated steering servo.
The steering error and the resulting differential PWM are computed by the following two functions, which implement the smoothing and proportional-control equations described above.

```
prev_error = 0
def compute_steering(left_fit, right_fit, shape):
    global prev_error
    h, w = shape[:2]
    y = h * 0.75                                    # look-ahead row
    lane_center = (np.polyval(left_fit, y) + np.polyval(right_fit, y)) / 2
    car_center  = w / 2
    error = lane_center - car_center
    error = 0.3 * prev_error + 0.7 * error        # temporal smoothing
    prev_error = error
    return error, lane_center, car_center

def compute_pwm(error, base_speed=135, max_adjust=130):
    error  = np.clip(error, -80, 80)
    adjust = (error / 80) * max_adjust            # range [-130, 130]
    left_pwm  = int(np.clip(base_speed + adjust, 0, 255))
    right_pwm = int(np.clip(base_speed - adjust, 0, 255))
    return left_pwm, right_pwm
```

Listing 4.4: Steering-error smoothing and proportional PWM controller (lane_following.py

4.3.10 Serial Communication with Arduino

The RPI computes steering commands and sends them to the Arduino through serial communication at 9600 baud.

Commands are formatted as:

L135R120

where:

L = Left Motor PWM
R = Right Motor PWM

Example:

L200R010

indicates a strong right turn.

Commands are transmitted at a maximum rate of:

20 Hz

(one command every 50 ms)

to prevent serial buffer overflow and ensure stable motor control.

The Arduino receives the PWM values and controls the L298N motor driver accordingly.

4.3.11 Lane Visualization

For debugging and performance evaluation, the detected lane area is projected back onto the original camera image using the inverse perspective matrix:

$$M^{-1}$$

The detected driving corridor is displayed as a green overlay.

Additional visual indicators include:

Green vertical line → Lane center.
Red vertical line → Vehicle center.
Steering direction text.
Current steering error value.
This visualization greatly assists system calibration and testing.

4.3.12 Lane Loss Fail-Safe Mechanism

A safety mechanism is implemented to prevent uncontrolled vehicle motion when lane markings are lost.

If lane detection fails:

left_fit == None

and

right_fit == None

the RPI immediately transmits:

L000R000

to the Arduino.

This causes:

Immediate vehicle stop.
Prevention of unintended steering.
Increased operational safety.

The fail-safe mechanism ensures the autonomous vehicle never continues moving blindly without valid lane information.
The main loop applies this logic directly: valid lane fits produce a rate-limited L###R### command, while a lost lane immediately transmits the L000R000 stop command.

```
if left_fit is not None or right_fit is not None:
    error, lane_center, car_center = compute_steering(left_fit, right_fit, frame.shape)
    left_pwm, right_pwm = compute_pwm(error)
    if time.time() - last_send_time >= 0.05:           # rate limit ~20 Hz
        ser.write(f"L{left_pwm:03d}R{right_pwm:03d}\n".encode()); ser.flush()
        last_send_time = time.time()
else:
    # FAIL-SAFE: no lane detected -> stop the car
    if time.time() - last_send_time >= 0.05:
        ser.write(b"L000R000\n"); ser.flush()
        last_send_time = time.time()
```

Listing 4.5: Rate-limited serial output and lane-loss fail-safe (lane_following.py)

4.3.13 Lane Detection Performance

The proposed lane detection system demonstrated stable real-time operation on the RPI 5 platform.

Key characteristics include:

Real-time processing capability.
Low computational requirements.
Robust lane tracking.
Smooth steering behavior.
Recovery from partial lane loss.
Safe emergency stop functionality.

By combining perspective transformation, color segmentation, sliding-window tracking, polynomial curve fitting, and differential motor control, the lane detection module provides reliable path-following performance and serves as the foundation for higher-level autonomous driving features such as obstacle avoidance, adaptive cruise control, and autonomous parking.

4.4 Hardware Implementation

The hardware implementation of the autonomous vehicle prototype bridges the theoretical system architecture with the physical environment. This section details the assembly, power distribution, wiring, and justification for the selected components.

Figure 4.7 shows the overall hardware overview of the prototype, illustrating the main processing, sensing, and actuation components and how they are connected.

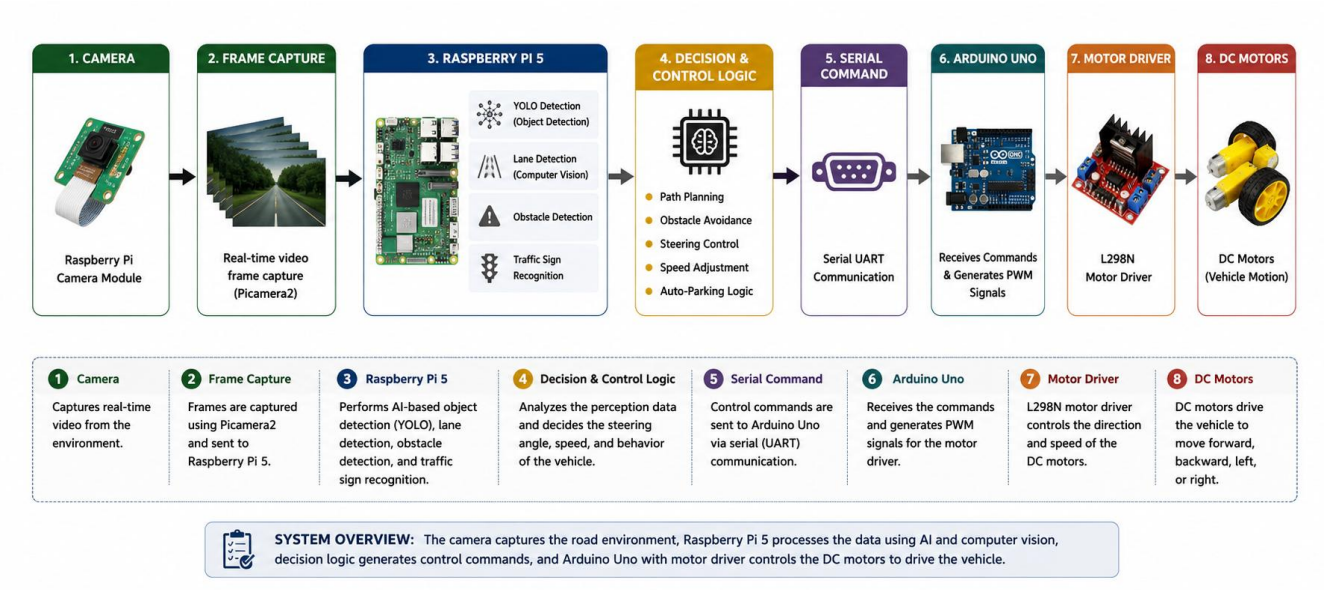


Figure 4.7: System Hardware Overview

4.4.1 Hardware Assembly and Physical Layout

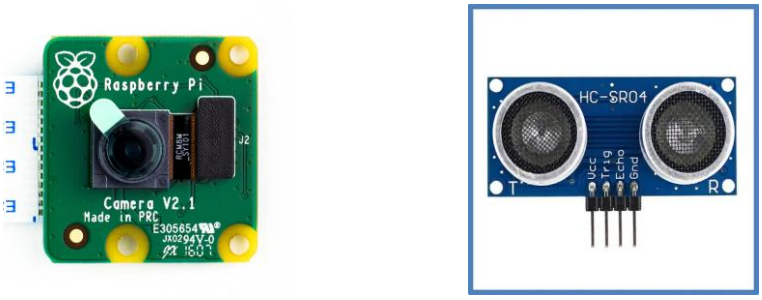
- Chassis and Mobility: The physical foundation of the prototype is a wheeled chassis.The mobility system relies on DC motors directly linked to the wheels and driven by an L298N motor driver module.

Figure 4.8 shows the assembled chassis of the prototype, which forms the physical foundation carrying the motors, wheels, and the rest of the hardware components.

Figure 4.8: Assembled Hardware Prototype

- Sensor Placement: To ensure accurate environmental perception, the RPI Camera Module is mounted at the front center of the chassis. It is positioned at an optimal height and angle to capture the field of view required for the BEV perspective transformation used in lane detection. The HC-SR04 ultrasonic sensor is also front-mounted to detect immediate obstacles and provide reliable proximity measurements.

Figure 4.9 shows the two main perception sensors used by the prototype: the RPI Camera Module and the HC-SR04 ultrasonic sensor.



RPI camera v2 Ultra sonic sensor

Figure 4.9: RPI Camera Module and HC-SR04 Ultrasonic Sensor

4.4.2 Power Distribution System

- Logic Power (5V/5A Power Bank): Dedicated to powering the RPI 5. The high current capacity (5A) is essential to sustain the heavy computational load of running the YOLOv8 object detection model and lane detection algorithms simultaneously without sudden power throttling or resets.

- Motor Power (7.4V Battery Pack): Strictly dedicated to the L298N motor driver and the DC motors.
- Arduino Power (7.4V Battery Pack): Strictly dedicated to Arduino uno (Vin).

4.4.3 Circuitry and Wiring Integration

The interaction between the processing and execution units relies on continuous, low-latency communication and physical wiring:

- **RPI to Arduino Communication:** The Raspberry Pi serves as the main processing unit, continuously analyzing camera frames using YOLO and lane detection algorithms. It computes steering and speed commands in real time and transmits them to the Arduino over a 9600 baud UART serial connection. Meanwhile, the Arduino continuously reports distance measurements from the ultrasonic sensor to the Raspberry Pi via the same serial link, enabling bidirectional communication and real-time obstacle awareness.

- Arduino to Motor Driver: The Arduino Uno R3 acts as the hardware execution unit. It receives the UART commands and translates them into electrical signals. Using specific General Purpose Input/Output (GPIO) pins, it sends Pulse Width Modulation (PWM) signals to the L298N motor driver to regulate the speed and direction of the DC motors. On the Arduino side, the firmware parses each L###R### command, extracts the two PWM values, and drives the motor channels accordingly, while also handling the auxiliary sweep and manoeuvre commands.

```
void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
      int lIndex = cmd.indexOf('L');
      int rIndex = cmd.indexOf('R');
      if (lIndex != -1 && rIndex != -1) {
        left_speed  = cmd.substring(lIndex + 1, rIndex).toInt();
        right_speed = cmd.substring(rIndex + 1).toInt();
        forward(left_speed, right_speed);
    }
  }
  static unsigned long lastSend = 0;
  if (millis() - lastSend > 100) {   // Every 100 ms
    lastSend = millis();
    distance = readDistanceCM();
    Serial.print("DIST:");
    Serial.println(distance);
  }
}
```

Listing 4.6: Arduino serial command parsing and motor actuation (control.ino)

4.5 Autonomous Driving State Machine

The autonomous behaviour of the prototype is coordinated by a finite-state machine (FSM) that runs once per camera frame. Rather than handling lane following, stop signs, obstacle avoidance, lane changing, and U-turns with a single block of conditional logic, each behaviour is encapsulated in its own state class. This keeps the per-frame decision logic readable, makes transitions explicit, and allows new manoeuvres to be added without destabilising the existing ones.

Every state implements a common interface with three methods: on_enter (executed once when the state becomes active), update (executed every frame, returning the name of the next state or None to remain), and on_exit (executed once when the state is left). A central controller holds a registry of all state instances and delegates the per-frame call to whichever state is currently active.

```
class State:
    def on_enter(self, car):
        pass

    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        """Called every frame. Returns the next state name, or None."""
        raise NotImplementedError

    def on_exit(self, car):
        pass
```

Listing 4.7: Base State interface for the driving state machine (states.py)

The CarController class owns the state registry and the shared vehicle context (current lane, sensor readings, detection flags, cooldown timers, and the output motor speeds). On each frame it forwards the update call to the active state and performs the transition if a new state name is returned.

```
class CarController:
    def __init__(self):
        self.hardware = hardware.SerialController()
        self.state_instances = {
            'LANE_FOLLOW': states.LaneFollowState(),
            'STOP': states.StopState(),
            'RED_LIGHT_WAIT': states.RedLightWaitState(),
            'STOP_SIGN_WAIT': states.StopSignWaitState(),
            'UTURN_STOP1': states.UTurnStop1State(),
            'UTURN_FORWARD': states.UTurnForwardState(),
            'UTURN_STOP2': states.UTurnStop2State(),
            'UTURN_STEER': states.UTurnSteerState(),
            'UTURN_STOP_FINAL': states.UTurnStopFinalState(),
            'REACH_LEFT_LANE_CENTER': states.ReachP1State(),
            'FORWARD': states.ForwardState(),
            'PAUSE': states.PauseState(),
            'STEER_RIGHT': states.SteerRightState(),
            'PAUSE_AFTER_STEER': states.PauseAfterSteerState(),
            'FORWARD_AFTER_STEER': states.ForwardAfterSteerState()
            'REACH_RIGHT_LANE_CENTER': states.ReachP1RightState(),
            'FORWARD_RIGHT': states.ForwardRightState(),
            'PAUSE_RIGHT': states.PauseRightState(),
            'STEER_LEFT_R': states.SteerLeftRState(),
            'PAUSE_AFTER_STEER_RIGHT': states.PauseAfterSteerRightState(),
            'FORWARD_AFTER_STEER_RIGHT': states.ForwardAfterSteerRightState(),
```

```
            'PARK_SEARCH': states.ParkSearchState(),
            'PARK_ALIGN': states.ParkAlignState(),
            'PARK_STOP1': states.ParkStop1State(),
            'PARK_BACK_STEER': states.ParkBackSteerState(),
            'PARK_BACK_STRAIGHT': states.ParkBackStraightState(),
            'PARK_COMPLETE': states.ParkCompleteState()

        }
        self.current_state_name = 'LANE_FOLLOW'
        self.active_state = self.state_instances[self.current_state_name]
        self.current_lane = 'RIGHT'
        self.state_start_time = time.time()
```

Listing 4.8: State registry and shared vehicle context (main.py)

State transitions are centralised in a single change_state method. It invokes the exit hook of the outgoing state, swaps the active state, resets the state timer (used by all timed manoeuvres), and invokes the entry hook of the incoming state. Logging each transition makes the vehicle's behaviour easy to trace during field testing.

```
def change_state(self, new_state_name):
    if new_state_name not in self.state_instances:
        print(f"Warning: state {new_state_name} does not exist!")
        return
    print(f"State Transition: {self.current_state_name} -> {new_state_name}")
    self.active_state.on_exit(self)
    self.current_state_name = new_state_name
    self.active_state = self.state_instances[new_state_name]
    self.state_start_time = time.time()
    self.active_state.on_enter(self)

def update(self, frame, left_fit, right_fit, left_valid, right_valid, mask):
    self.both_blocked_slow_down = False
    next_state_name = self.active_state.update(
        self, frame, left_fit, right_fit, left_valid, right_valid, mask)
    if next_state_name is not None:
        self.change_state(next_state_name)
```

Listing 4.9: Centralised state-transition logic (main.py)

The default state is LANE_FOLLOW. From this state the vehicle continuously evaluates a prioritised set of conditions: a detected lane end triggers a U-turn; the disappearance of both lane boundaries triggers a safety stop; a stop sign or red light within range triggers a timed stop; and an obstacle in the current lane triggers either a lane change or a slow-down, depending on whether the adjacent lane is clear. If none of these conditions hold, the vehicle simply steers to keep itself centred in the lane.

```
class LaneFollowState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        now = time.time()

        # 1. Lane end (horizontal line) -> U-Turn
        if detect_lane_end(mask) and (now > car.uturn_cooldown_until):
            return 'UTURN_FORWARD'

        # 2. Both lane boundaries lost -> Stop
```

```
        if left_fit is None and (now > car.uturn_cooldown_until):
            return 'STOP'

        # 3. Stop sign within range -> timed stop
        if (car.stop_sign_inside or car.stop_sign_outside) and \
           (car.min_dist <= 40.0 or car.distance <= 40.0) and \
           (now > car.ignore_stop_until):
            car.stop_until = now + 3.0
            car.ignore_stop_until = now + 8.0
            car.skip_detection_until = now + 8.0
            return 'STOP_SIGN_WAIT'

        # 4. Red traffic light -> timed stop
        if car.red_light_outside and (now > car.ignore_stop_until):
            print("🔘 Traffic red light detected! Stopping until GREEN light is
            detected...")
            return 'RED_LIGHT_WAIT'


        # 5. Obstacle handling (see Section 4.8)
        # ... lane-change / slow-down decision ...

        # Standard steering control
        if left_fit is not None:
            error, lane_center, car_center = compute_steering(
                left_fit, right_fit, left_valid, right_valid,
                frame.shape, 'LANE_FOLLOW')
            car.left_pwm, car.right_pwm = compute_pwm(error)
        return None
```

Listing 4.10: Prioritised decision logic of the lane-following state (states.py)

This layered prioritisation guarantees that safety-critical events (lane end, lost lanes, stop signs, and red lights) are always evaluated before comfort-oriented behaviours such as lane changing, and that ordinary steering is only applied when no higher-priority event is active.

4.5.1 Lane-Change Maneuver

When an obstacle blocks the vehicle's current lane while the adjacent lane is clear, the system performs an automated lane change. The manoeuvre is implemented as a short, ordered sequence of timed sub-states rather than as a single continuous control law, because the camera briefly loses a reliable view of both lane boundaries during the transition. Breaking the manoeuvre into discrete phases keeps it predictable and tunable.

The sequence proceeds as follows: REACH_RIGHT_LANE_CENTER or REACH_LEFT_LANE_CENTER steers the vehicle toward the center of the target lane, depending on the selected lane-change direction. Once the vehicle reaches the target lane center, FORWARD drives straight to complete the lane transition. PAUSE briefly stabilises the vehicle and camera image. Next, STEER_RIGHT or STEER_LEFT aligns the vehicle with the new lane until both lane boundaries are reliably detected. Finally, PAUSE_AFTER_STEER and FORWARD_AFTER_STEER allow the vehicle to settle before normal lane-following resumes in the newly selected lane.

The first phase, REACH_LEFT_LANE_CENTER, reuses the standard steering controller but feeds the center of the target lane instead of the detected current lane centre, biasing the vehicle toward the adjacent lane. The phase ends after a fixed duration.

```
class ReachLeftLaneCenterState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        error, lane_center, car_center = compute_steering(
            left_fit, right_fit, left_valid, right_valid,
            frame.shape, 'REACH_LEFT_LANE_CENTER')
        car.command = get_command(error)
        car.left_pwm, car.right_pwm = compute_pwm(error)
        if time.time() - car.state_start_time >= config.LeftCenter_DURATION:
            return 'FORWARD'
        return None
```

Listing 4.11: First phase of the lane change toward the center of the target lane (states.py)

The steering phase, STEER_RIGHT, applies an asymmetric motor command that pivots the vehicle into the new lane. It continues until the lane detector reports both boundaries of the destination lane as valid, at which point the vehicle has aligned with the new lane and the manoeuvre proceeds to its settling phases.

```
class SteerRightState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 195
        car.right_pwm = 0
        car.command = "STEER_RIGHT"
        if left_valid and right_valid:
            return 'PAUSE_AFTER_STEER'
        return None

class ForwardAfterSteerState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 180
        car.right_pwm = 180
        car.command = "FORWARD"
        if time.time() - car.state_start_time >= \
                config.FORWARD_AFTER_STEER_DURATION:
            car.current_lane = 'LEFT'
            return 'LANE_FOLLOW'
        return None
```

Listing 4.12: Steering into the new lane and resuming lane following (states.py)

After lane boundaries are detected, each obstacle is localized relative to the road by projecting the bottom-center point of its bounding box into the bird's-eye view using the perspective transformation matrix. This point represents the obstacle's contact position with the road surface. The transformed point is then compared with the left and right lane boundaries obtained from the fitted lane polynomials. If the point lies between the two boundaries, the obstacle is classified as being in the vehicle's current lane. Otherwise, its position is evaluated relative to the adjacent lane depending on whether the vehicle is currently driving in the left or right lane. The lane width estimated from the detected boundaries is used to determine whether the obstacle belongs to the neighboring lane or is outside the drivable road area. According to this classification, obstacle flags

are updated for each lane, enabling the decision-making module to determine whether a lane change is required or whether the adjacent lane is safe for maneuvering.

```
# Lane localization for physical obstacles
if  left_fit is not None and right_fit is not None:
    x_center = (x1 + x2) / 2.0
    y_bottom = float(y2)
    pts = np.array([[[x_center, y_bottom]]], dtype=np.float32)
    pts_warped = cv2.perspectiveTransform(pts, M)
    xw, yw = pts_warped[0][0]

    x_left = np.polyval(left_fit, yw)
    x_right = np.polyval(right_fit, yw)
    lane_width_pixels = x_right - x_left

    # 1. Obstacle inside car's current lane
    if x_left <= xw <= x_right:
        object_lane_tag = f"IN MY LANE ({car.current_lane})"
        car.has_obstacle = True
        if car.current_lane == 'RIGHT':
            car.right_lane_has_obstacle = True
        else:
            car.left_lane_has_obstacle = True

    elif car.current_lane == 'RIGHT':
        # In RIGHT lane: Left side (x_left) is middle line, Right side (x_right) is road
        edge
        if (x_left - lane_width_pixels ) <= xw < x_left:
            object_lane_tag = "IN LEFT LANE"
            car.left_lane_has_obstacle = True
        else:
            object_lane_tag = "OUTSIDE ROAD"

    elif car.current_lane == 'LEFT':
        # In LEFT lane: Left side (x_left) is road edge, Right side (x_right) is middle
        line
        if x_right < xw <= (x_right + lane_width_pixels ):
            object_lane_tag = "IN RIGHT LANE"
            car.right_lane_has_obstacle = True
        else:
            object_lane_tag = "OUTSIDE ROAD"
```

Listing 4.13: Obstacle lane localisation using the bird's-eye-view lane model (main.py)

4.5.2 U-Turn Maneuver

The U-turn manoeuvre allows the vehicle to reverse its direction of travel when it reaches the end of the drivable lane. It is triggered automatically when the lane detector identifies a horizontal line across the road, which marks the end of the lane. Like the lane change, the U-turn is decomposed into a chain of timed sub-states that together produce a controlled half-rotation.

The trigger itself is the detect_lane_end function. It combines three independent cues so that a lane end is only declared when the evidence is strong: a row-projection test that counts bright pixels per image row, a contour test that looks for a wide, low-aspect-ratio white blob, and a Hough-line test that searches for a long, near-horizontal line. If any cue fires, the lane end is confirmed.

```
def detect_lane_end(binary_img):
```

```
    h, w = binary_img.shape
    roi = binary_img

    # 1. Row-sum projection
    row_sums = np.sum(roi == 255, axis=1)
    max_row_sum = np.max(row_sums) if len(row_sums) > 0 else 0

    # 2. Contours (wide, low-aspect blob)
    contours, _ = cv2.findContours(roi, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    lane_end_by_contour = False
    for c in contours:
        if cv2.contourArea(c) > 2500:
            x, y, wb, hb = cv2.boundingRect(c)
            if wb > 170 and (wb / hb) > 0.45:
                lane_end_by_contour = True
                break

    if max_row_sum > 150 or lane_end_by_contour:
        return True

    # 3. Hough lines (long, near-horizontal)
    lines = cv2.HoughLinesP(roi, 1, np.pi / 180, threshold=30,
                            minLineLength=100, maxLineGap=40)
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            slope = (y2 - y1) / ((x2 - x1) + 1e-6)
            length = np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
            if abs(slope) < 0.25 and length > 120:
                return True
    return False
```

Listing 4.14: Lane-end detection that triggers the U-turn (real_lane.py)

Once a lane end is confirmed, the controller enters the U-turn chain. The vehicle first drives forward a short distance to clear the line, stops to stabilise, then applies a sustained left-pivot command. During the pivot the lane detector runs continuously; as soon as both boundaries of the opposite lane become valid, the manoeuvre stops and normal lane following resumes. A timeout guards against the case where the lane is never re-acquired.

```
class UTurnForwardState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 180
        car.right_pwm = 180
        car.command = "UTURN_FORWARD"
        if time.time() - car.state_start_time >= 1.1:
            return 'UTURN_STOP2'
        return None

class UTurnSteerState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 190
        car.command = "UTURN_STEER"
        elapsed = time.time() - car.state_start_time
        if elapsed >= config.UTURN_MIN_STEER_DURATION:
```

```
            if left_valid and right_valid:
                return 'UTURN_STOP_FINAL'
            elif elapsed >= config.UTURN_TIMEOUT:
                return 'LANE_FOLLOW'
        return None
```

Listing 4.15: Forward and steering phases of the U-turn (states.py)

After the pivot completes, the final phase resets the active lane to the right lane and arms a cooldown timer. The cooldown prevents the freshly detected opposite lane line (and the line the vehicle just crossed) from immediately re-triggering another U-turn, which would otherwise leave the vehicle spinning in place.

```
class UTurnStopFinalState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "UTURN_STOP"
        now = time.time()
        if now - car.state_start_time >= 1.0:
            car.current_lane = 'RIGHT'
            car.uturn_cooldown_until = now + 8.0
            return 'LANE_FOLLOW'
        return None
```

Listing 4.16: Final U-turn phase with lane reset and cooldown (states.py)

4.5.3 Obstacle Detection and Avoidance Logic

Obstacle avoidance fuses two independent sources of information: the YOLO object detector, which provides the type and image position of objects ahead, and the ultrasonic, which provides a direct distance measurement. Combining vision with range data makes the system robust: vision identifies what an object is and which lane it occupies, while the ultrasonic sensor confirms how close it actually is.

Each detected object is first projected into the BEV and tested against the fitted lane polynomials to decide whether it lies inside the current lane, in the adjacent lane, or off-road. This per-lane localisation is what allows the controller to distinguish between an obstacle it must avoid and one it can safely ignore.

```
def is_inside_lane(box, left_fit, right_fit, M):
    if left_fit is None or right_fit is None:
        return False
    x1, y1, x2, y2 = box
    x_center = (x1 + x2) / 2.0
    y_bottom = float(y2)
    pts = np.array([[[x_center, y_bottom]]], dtype=np.float32)
    pts_warped = cv2.perspectiveTransform(pts, M)
    xw, yw = pts_warped[0][0]
    x_left = np.polyval(left_fit, yw)
    x_right = np.polyval(right_fit, yw)
    return x_left <= xw <= x_right
```

Listing 4.17: Projecting a detection into the lane to test occupancy (vision_utils.py)

A coarse distance is also estimated directly from each detection's bounding-box area, exploiting the inverse relationship between apparent size and distance. This vision-based estimate complements the ultrasonic reading and provides a fallback when the sensor is not pointing at the object.

```
def estimate_distance(area):
    if area <= 0:
        return 10.0
    # Larger box area -> closer object
    return (2000 / area) * 100
```

Listing 4.18: Bounding-box-area distance estimation (vision_utils.py)

With each object localised to a lane and a distance, the decision logic compares the occupancy of the current and adjacent lanes. If the current lane is blocked but the other lane is clear, the controller initiates a lane change in the appropriate direction. If both lanes are blocked, it slows down at moderate range and stops at close range. The relevant portion of the lane-following state is shown below.

```
# Obstacle handling inside LaneFollowState.update()
if car.has_obstacle:
    if car.current_lane == 'RIGHT':
        current_lane_blocked = car.right_lane_has_obstacle
        other_lane_blocked = car.left_lane_has_obstacle
    else:
        current_lane_blocked = car.left_lane_has_obstacle
        other_lane_blocked = car.right_lane_has_obstacle

    if current_lane_blocked:
        if other_lane_blocked:
            # Both lanes blocked
            if car.min_dist <= 10.0 or car.distance <= 10.0:
                return 'STOP'
            elif car.min_dist <= 50.0 or car.distance <= 50.0:
                car.both_blocked_slow_down = True
        else:
            # Adjacent lane is clear -> change lane
            if car.current_lane == 'RIGHT':
                if car.min_dist <= 35.0 or car.distance <= 35.0:
                    return 'REACH_LEFT_LANE_CENTER'
            else:
                print(f" Obstacle in LEFT lane. RIGHT lane is empty. Changing lane RIGHT!")
                if car.min_dist <= 35.0 or car.distance <= 35.0:
                    return 'REACH_RIGHT_LANE_CENTER'
```

Listing 4.19: Rule-based obstacle-avoidance decision (states.py)

The per-frame object loop in the main orchestrator classifies each detection by type and updates the corresponding context flags. Stop signs and red lights are handled according to whether they fall inside the lane, while a yellow light inside the lane arms a temporary slow-down. The same loop records which lane each obstacle occupies, feeding the decision logic above.

```
for detection in detections:
    name = detection["name"]
    x1, y1, x2, y2 = detection["box"]
    det_dist = vision_utils.estimate_distance(detection["area"])
```

```
    car.distance = min(car.distance, det_dist)
    in_lane = vision_utils.is_inside_lane((x1, y1, x2, y2),
                                          left_fit, right_fit, M)
    if name == "stop-sign":
        if in_lane: car.stop_sign_inside = True
        else:       car.stop_sign_outside = True
    elif name == "red":
        if not in_lane: car.red_light_outside = True
    elif name == "yellow":
        if in_lane:
            car.yellow_slow_until = time.time() + \
                config.YELLOW_SLOW_DURATION
```

Listing 4.20: Per-detection classification and flagging (main.py)

4.5.4 Distance Sensing and Serial Telemetry

The RPI and the Arduino communicate over a single 9600-baud serial link. The Pi sends motor commands and high-level instructions; the Arduino streams back ultrasonic distance telemetry. To keep the perception loop responsive, all serial reads on the Pi are non-blocking and all writes are rate-limited and protected by an automatic reconnection mechanism.

Distance telemetry is read by draining whatever the Arduino has sent since the previous frame and parsing the most recent DIST: line. Because the read never blocks, a slow or silent sensor cannot stall the vision pipeline.

```
def read_distance(self, default_dist=999.0):
    if self.ser is None or not self.ser.is_open:
        return default_dist
    latest_dist = default_dist
    try:
        while self.ser.in_waiting > 0:
            line = self.ser.readline().decode(
                'utf-8', errors='ignore').strip()
            if "DIST:" in line:
                try:
                    latest_dist = float(line.split(":")[1].strip())
                except ValueError:
                    pass
    except Exception as e:
        print(f"Error reading serial telemetry: {e}")
    return latest_dist
```

Listing 4.21: Non-blocking distance telemetry reader (hardware.py)

Motor commands are encoded in the compact LxxxRxxx format, where the two three-digit fields carry the left and right PWM duty cycles. Writes are throttled to roughly 20 Hz to avoid flooding the serial buffer, and a guarded write routine transparently re-opens the port if the connection drops mid-drive.

```
def send_motor_speeds(self, left_pwm, right_pwm,
                      current_state, min_dist, force=False):
    current_time = time.time()
    # Rate limit to 20 Hz (every 50 ms) unless forced
    if force or (current_time - self.last_send_time >= 0.05):
        command_str = f"L{int(left_pwm):03d}R{int(right_pwm):03d}\n"
        written = self.safe_write(command_str.encode())
```

```
        if written:
            self.last_send_time = current_time
```

Listing 4.22: Rate-limited motor-command transmission (hardware.py)

Together these modules form a complete, real-time autonomous-driving stack: the vision pipeline of Section 4.3 perceives the lane, the YOLO module of Section 4.2 recognises objects, the state machine of Section 4.5 selects the appropriate behaviour, the lane-change and U-turn manoeuvres of Sections 4.6 and 4.7 execute complex movements, the obstacle logic of Section 4.8 keeps the vehicle safe, and the telemetry layer of this section binds the high-level decisions to the embedded motor controller.

4.5.5 Real-Time Detection and Inference Pipeline

The object-detection module wraps the YOLOv8n model exported to ONNX and runs it through the Ultralytics runtime [5]. Frames are captured directly from the RPI camera, colour-corrected, and letterbox-resized to the network's input size before inference. Keeping the camera handling and the detector in a single module allows the rest of the system to request a fresh, fully processed frame with a single call.

```
def init_detector():
    global picam2, model
    if model is None:
        model = YOLO(config.MODEL_PATH, task="detect")
    if picam2 is None:
        picam2 = Picamera2()
        picam2.configure(
            picam2.create_preview_configuration(
                main={"size": config.TARGET_SIZE}))
        picam2.start()
        print("Camera started")
        print("Model loaded")
```

Listing 4.23: Camera and YOLO model initialisation (object_detection.py)

Although the single-pass design of YOLO [4] already suppresses most duplicate predictions, a custom non-maximum suppression (NMS) routine is applied as a safeguard. It sorts candidate boxes by confidence and greedily removes any lower-confidence box whose intersection-over-union with a kept box exceeds a fixed threshold, leaving one clean detection per object.

```
def nms(boxes, scores, iou_threshold=0.45):
    boxes = np.array(boxes); scores = np.array(scores)
    indices = scores.argsort()[::-1]
    keep = []
    while len(indices) > 0:
        i = indices[0]
        keep.append(i)
        if len(indices) == 1:
            break
        x1 = np.maximum(boxes[i][0], boxes[indices[1:], 0])
        y1 = np.maximum(boxes[i][1], boxes[indices[1:], 1])
        x2 = np.minimum(boxes[i][2], boxes[indices[1:], 2])
        y2 = np.minimum(boxes[i][3], boxes[indices[1:], 3])
        inter = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)
        area_i = (boxes[i][2]-boxes[i][0])*(boxes[i][3]-boxes[i][1])
```

```
        area_o = ((boxes[indices[1:],2]-boxes[indices[1:],0]) *
                  (boxes[indices[1:],3]-boxes[indices[1:],1]))
        iou = inter / (area_i + area_o - inter + 1e-6)
        indices = indices[1:][iou < iou_threshold]
    return keep
```

Listing 4.24: Custom non-maximum suppression (object_detection.py)

The detection function ties these pieces together. It captures a frame, fixes the colour channels, resizes while preserving aspect ratio, runs the YOLO model at a fixed confidence threshold, applies NMS, and finally returns a list of structured detections, each carrying the class name, confidence, bounding box, and pixel area used later for distance estimation.

```
def detect_objects(skip_yolo=False):
    if picam2 is None:
        init_detector()
    frame = picam2.capture_array()
    if frame.shape[2] == 4:
        frame = cv2.cvtColor(frame, cv2.COLOR_RGBA2BGR)
    else:
        frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
    h0, w0 = frame.shape[:2]
    r = min(config.TARGET_SIZE[0]/w0, config.TARGET_SIZE[1]/h0)
    frame_resized = cv2.resize(frame, (int(w0*r), int(h0*r)))
    if skip_yolo:
        return frame_resized, []
    results = model(frame_resized, conf=0.5)
    boxes_all, scores_all, class_ids_all = [], [], []
    if results[0].boxes is not None:
        for box in results[0].boxes:
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            boxes_all.append([x1, y1, x2, y2])
            scores_all.append(float(box.conf[0]))
            class_ids_all.append(int(box.cls[0]))
    keep = nms(boxes_all, scores_all)
    detections = []
    for i in keep:
        x1, y1, x2, y2 = map(int, boxes_all[i])
        detections.append({
            "name": results[0].names[class_ids_all[i]],
            "conf": scores_all[i],
            "box": (x1, y1, x2, y2),
            "area": (x2 - x1) * (y2 - y1)})
    return frame_resized, detections
```

Listing 4.25: Full detection routine with NMS and structured output (object_detection.py)

4.5.6 Main Orchestration Loop

All of the modules described in this chapter are coordinated by a single real-time loop. Each iteration reads the latest ultrasonic distance, captures and processes a camera frame, runs lane detection and object detection, updates the per-lane obstacle flags, advances the state machine, renders an on-screen telemetry overlay, and finally transmits the resulting motor command to the Arduino. The loop is deliberately structured so that perception, decision, and actuation happen in a fixed, predictable sequence every frame.

```
while True:
    # 1. Telemetry
    car.min_dist = car.hardware.read_distance(default_dist=car.min_dist)
    car.distance = 999.0

    # 2. Object detection
    frame, detections = detect_objects()

    # 3. Lane detection
    warped, Minv, M, roi_debug = perspective_transform(frame)
    mask = threshold_white(warped)
    left_fit, right_fit, left_valid, right_valid = fit_polynomial(mask)

    # 4. Localise detections to lanes (Section 4.8)
    # ... update car.stop_sign_*, car.red_light_*, lane obstacle flags ...

    # 5. Decision: advance the state machine (Section 4.5)
    car.update(frame, left_fit, right_fit, left_valid, right_valid, mask)

    # 6. Render overlay and 7. send motor command
    car.send_controls()

    if cv2.waitKey(1) & 0xFF == 27:   # ESC to exit
        break
```

Listing 4.26: Real-time perception-decision-actuation loop (main.py)

The on-screen overlay drawn each frame reports the active state, the current lane, the commanded wheel speeds, the steering error, and the occupancy status of the left and right lanes. During development this overlay was the primary debugging tool, making the otherwise invisible internal state of the controller directly observable while the vehicle was driving.

```
cv2.putText(result, f"State: {state_text}", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
cv2.putText(result, f"Lane: {display_lane}", (20, 80),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, lane_color, 2)
cv2.putText(result, f"Speed: L{car.left_pwm} R{car.right_pwm}",
            (20, 120), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
cv2.putText(result, f"LEFT Lane: {left_lane_status}", (560, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, left_color, 2)
cv2.putText(result, f"RIGHT Lane: {right_lane_status}", (560, 80),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, right_color, 2)
```

Listing 4.27: Telemetry overlay for live debugging (main.py)

The complete software stack therefore integrates four cooperating layers in line with the integrated-system goal of this project: a perception layer built on classical lane detection and YOLO-based object detection [4], a decision layer expressed as the finite-state machine of Section

4.5, a manoeuvre layer providing lane changing and U-turns, and an actuation layer that fuses camera and ultrasonic information [8] and drives the embedded motor controller. This unified design distinguishes the prototype from systems that address only a single sub-task in isolation.

4.5.7 Embedded Motor Control and Calibration

The Arduino firmware is responsible for translating the high-level commands received from the RPI into the low-level signals that drive the L298N motor driver. Each motor channel is controlled by a direction pair (IN1/IN2 for the left motor and IN3/IN4 for the right) and a PWM enable pin (enA and enB). Setting the direction pins selects forward rotation, while the analog write on the enable pin sets the speed. This clean separation between direction and speed makes the higher-level differential-drive commands straightforward to realise.

```
int IN1 = 4, IN2 = 10;   // left motor direction
int IN3 = 6, IN4 = 7;    // right motor direction
int enA = 3, enB = 5;    // left / right PWM enable

void forward(int leftSpeed, int rightSpeed)
{
    if (leftSpeed >= 0) {
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, HIGH);
        analogWrite(enB, leftSpeed);
    } else {
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
        analogWrite(enB, abs(leftSpeed));
    }
    if (rightSpeed >= 0) {
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(enA, rightSpeed);
    } else {
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
        analogWrite(enA, abs(rightSpeed));
    }
}

void stopCar() {
  analogWrite(enA, 0);
  analogWrite(enB, 0);
}
```

Listing 4.28: Differential-drive motor primitives on the Arduino (control.ino)

The ultrasonic distance reading is performed with the standard trigger-echo timing method. A short pulse is emitted on the trigger pin, and the duration of the returning echo is measured and converted to a distance using the speed of sound. A timeout guards against missing echoes, in which case a large sentinel distance is returned so that the absence of an obstacle is never mistaken for a close one.

```
float readDistance() {
  digitalWrite(trigPin, LOW);  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
```

```
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) return 999;       // no echo -> far
  distance = duration * 0.034 / 2;     // speed of sound
  return distance;
}
```

Listing 4.29: Ultrasonic distance measurement (control.ino)

The fusion of camera and ultrasonic sensing implemented across these modules follows the multisensor obstacle-detection principle described by Stiller et al. [8]: independent sensing modalities with overlapping fields of view yield a more reliable estimate of the surrounding environment than any single sensor could provide on its own. In this prototype the camera supplies object identity and lane occupancy while the swept ultrasonic sensor supplies a direct, vision-independent distance measurement.

Several of the manoeuvre parameters in config.py were obtained empirically through a dedicated calibration utility. This helper script streams the camera feed, overlays the candidate lane-change target points, and lets the developer issue motor commands interactively while observing the vehicle's response. The values that produced smooth, repeatable manoeuvres were then frozen into the configuration constants used by the state machine.

```
# Tuning constants frozen after empirical calibration
STOP_SIGN_AREA_THRESHOLD = 25000
YELLOW_SLOW_DURATION = 4.0
UTURN_MIN_STEER_DURATION = 0.0
UTURN_TIMEOUT = 6.5
TARGET_SIZE = (800, 600)
LANE_WIDTH = 580   # expected distance between lines (pixels)
# User Custom Auto Parking Settings (Right-Side Reverse Parking)
PARK_SEARCH_DIST_THRESHOLD = 30.0  # Distance threshold (> 30 cm)
PARK_SEARCH_DURATION = 0.3         # Space verification time (0.3 s)
PARK_ALIGN_DURATION = 0.6          # Forward positioning time past the spot (0.6 s)
PARK_BACK_STEER_DURATION = 1.2     # Reverse turn right into spot (1.2 s)
PARK_BACK_STRAIGHT_DURATION = 0.6   # Reverse straight adjustment (0.6 s)
PARK_STOP_DURATION = 0.3           # Pause duration between movements
# Motor Speeds for maneuvers (LxxxRxxx)
PARK_SEARCH_SPEED = 140
PARK_ALIGN_SPEED = 150
PARK_BACK_STEER_LEFT = -190        # Left motor reverses fast to swing tail right
PARK_BACK_STEER_RIGHT = -10        # Right motor reverses slow
PARK_BACK_STRAIGHT = -160          # Both motors reverse straight
```

Listing 4.30: Calibrated system tuning constants (config.py)

Together, the embedded firmware and the calibration workflow close the loop between the high-level perception and decision software on the RPI and the physical actuation of the vehicle. The result is a complete, reproducible pipeline in which every tunable parameter has a defined meaning and a documented value, making the system straightforward to re-calibrate for a different chassis or track.

4.5.8 Traffic-Sign and Traffic-Light Response

Beyond avoiding physical obstacles, the prototype reacts to traffic-control elements detected by the YOLO model: stop signs and red, yellow, and green traffic lights. Each element produces a different behaviour, and the response depends not only on the detected object class but also on whether it lies within the vehicle's lane and how close it is, combining the detection identity from YOLO [4] with the distance estimate from the fused sensing layer [8].

When a stop sign is detected within the danger distance, the vehicle performs a timed full stop. The controller records a deadline three seconds in the future and transitions into a dedicated waiting state. To prevent the same sign from being processed repeatedly after the vehicle resumes, two cooldown timers are activated: one suppresses further stop-sign events, while the other temporarily skips object detection.

When a red traffic light is detected within the danger distance, the vehicle comes to a complete stop and remains stationary for as long as the red light continues to be detected. Unlike the stop-sign behaviour, no fixed waiting time is used. Instead, the controller continuously monitors the traffic light state and resumes driving only after a green traffic light is detected, ensuring compliance with the traffic signal.

```
# Inside LaneFollowState.update()
if car.stop_sign_outside and \
   (car.min_dist <= 40.0 or car.distance <= 40.0) and \
   (now > car.ignore_stop_until):
    car.stop_until = now + 3.0
    car.ignore_stop_until = now + 8.0
    car.skip_detection_until = now + 8.0
    return 'STOP_SIGN_WAIT'

if car.red_light_outside and (now > car.ignore_stop_until):
            print("🔘 Traffic red light detected! Stopping until GREEN light is
            detected...")
            return 'RED_LIGHT_WAIT'
```

Listing 4.31: Stop-sign and red-light triggering logic (states.py)

The waiting state itself simply holds the vehicle stationary until the recorded deadline passes, after which control returns to ordinary lane following. Keeping the timed wait in its own state means the rest of the system continues to run normally — telemetry is still read and the display is still updated — while the vehicle is paused.

```
class StopSignWaitState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "STOPPED"
        if time.time() >= car.stop_until:
```

```
            return 'LANE_FOLLOW'
        return None

class RedLightWaitState(State):
    def update(self, car, frame, left_fit, right_fit, left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "RED_LIGHT_STOP"
        if car.green_light_detected:
            print("🟢 Traffic green light detected! Resuming lane following.")
            car.ignore_stop_until = time.time() + 4.0
            return 'LANE_FOLLOW'
        return None
```

Listing 4.32: Timed stop-and-wait state & red light state (states.py)

A yellow light is treated as a caution rather than a halt. When a yellow light is detected inside the lane, the controller arms a short slow-down window. While that window is active and an object remains within range, the steering controller is invoked with a reduced base speed, producing a gentle deceleration instead of a stop. This graduated response mirrors the rapid, proportionate decision-making highlighted in collision-avoidance research [7].

```
# Yellow light arms a temporary slow-down window
elif name == "yellow":
    if in_lane:
        car.yellow_slow_until = time.time() + \
            config.YELLOW_SLOW_DURATION

# Later, in the steering branch of LaneFollowState:
slow_mode = now < car.yellow_slow_until
if (slow_mode or car.both_blocked_slow_down) and \
   (car.distance <= 60.0 or car.min_dist <= 60.0):
    car.left_pwm, car.right_pwm = compute_pwm(error, base_speed=100)
    car.command = get_command(error) + " (SLOW)"
else:
    car.left_pwm, car.right_pwm = compute_pwm(error)
```

Listing 4.33: Yellow-light caution and slow-down behaviour (states.py)

The safety-stop state provides the final layer of protection. If one lane boundary vanishes or both lanes are blocked at very close range, the vehicle enters a hard stop and remains there until the lanes reappear and the path ahead is clear beyond the safe threshold. Only when both conditions are satisfied does it resume lane following, ensuring the vehicle never drives blindly into an unmapped or blocked area.

```
class StopState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "STOPPED"
        if left_valid and right_valid and \
           car.min_dist > 60.0 and car.distance > 60.0:
            return 'LANE_FOLLOW'
        return None
```

Listing 4.34: Safety-stop state with clear-path resume condition (states.py)

Taken together, the responses in this section give the prototype a complete repertoire of reactions to the structured elements of its environment: it halts for stop signs and red lights, eases off for yellow lights, and refuses to move when the road ahead cannot be perceived safely. Combined with the obstacle-avoidance logic of Section 4.8 and the manoeuvres of Sections 4.6 and 4.7, these behaviours allow the single integrated controller to handle the full range of situations encountered on the test track.

4.6 Automatic Parking Maneuver

In addition to the driving and obstacle-handling behaviors described in Section 4.5, the prototype implements an automatic reverse-parking maneuver that allows the vehicle to detect a free parking slot on its right-hand side and park itself without any human intervention. The maneuver is fully integrated into the same finite-state machine that governs the rest of the system, so parking is expressed as an ordered chain of dedicated states rather than as a single monolithic routine. This keeps each phase of the maneuver short, independently tunable, and easy to reason about.

The parking behavior is decomposed into six sequential states: searching for a clear space, aligning the vehicle, pausing before the reverse turn, reversing while steering into the slot, straightening inside the slot, and finally holding the parked position. Each state performs one well-defined action and then hands control to the next state, exactly as the lane-change and U-turn maneuvers do. The complete sequence is summarized in Figure 4.10 and detailed in the subsections that follow.

Figure 4.10: State-transition flow of the automatic reverse-parking maneuver. Each box is a dedicated state in the finite-state machine; the maneuver advances only when the current state's exit condition is met.

All timing, speed, and steering parameters used by the parking states are defined as named constants in the configuration module. Centralising these values allows the maneuver to be calibrated for different chassis geometries, slot sizes, and floor surfaces without modifying the control logic itself.

```
# Automatic parking parameters (config.py)
PARK_SEARCH_SPEED          = 120     # forward speed while scanning
PARK_SEARCH_DIST_THRESHOLD = 30      # cm; space considered "clear"
PARK_SEARCH_DURATION       = 0.6     # s of continuous clearance required
PARK_ALIGN_SPEED           = 130     # forward nudge to position the slot
PARK_ALIGN_DURATION        = 0.5     # s
PARK_STOP_DURATION         = 0.4     # s pause before reversing
PARK_BACK_STEER_LEFT       = 150     # left wheel PWM during reverse turn
PARK_BACK_STEER_RIGHT      = 60      # right wheel PWM during reverse turn
PARK_BACK_STEER_DURATION   = 1.4     # s of reverse-and-turn
PARK_BACK_STRAIGHT         = 140     # straight reverse PWM
PARK_BACK_STRAIGHT_DURATION= 0.8     # s to settle inside the slot
```

Listing 4.35: Calibrated parameters for the automatic parking maneuver (config.py)

4.6.1 Parking-Space Search

The maneuver begins in the ParkSearch state. The vehicle drives slowly forward along the lane while continuously monitoring the right-side distance reported by the ultrasonic sensor. A candidate parking slot is recognised only when the measured clearance stays above a configurable threshold (30 cm) for a sustained period rather than for a single instantaneous reading. This debouncing prevents brief sensor spikes, gaps between objects, or noise from being mistaken for a genuine parking space.

A timer (space_clear_start) records the moment the clearance first exceeds the threshold. If the clearance is maintained for the full required duration, the state confirms the slot and transitions to alignment; if the clearance drops at any point, the timer is reset and the search continues.

```
class ParkSearchState(State):
    def on_enter(self, car):
        car.space_clear_start = None
        print("Auto Park: searching for right-side space (>30cm)...")

    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = config.PARK_SEARCH_SPEED
        car.right_pwm = config.PARK_SEARCH_SPEED
        car.command = "PARK_SEARCH"

        current_dist = min(car.min_dist, car.distance)
        if current_dist > config.PARK_SEARCH_DIST_THRESHOLD:
            if car.space_clear_start is None:
                car.space_clear_start = time.time()
            elif (time.time() - car.space_clear_start
                  >= config.PARK_SEARCH_DURATION):
                return 'PARK_ALIGN'      # slot confirmed
        else:
            car.space_clear_start = None # reset on any blockage
        return None
```

Listing 4.36: Parking-space search with sustained-clearance debouncing (states.py)

4.6.2 Alignment and Pre-Reverse Pause

Once a slot is confirmed, the vehicle must move slightly further forward so that its rear axle is correctly positioned relative to the slot entrance before reversing. The ParkAlign state drives forward for a short, fixed duration to achieve this offset. It is followed by the ParkStop1 state, which brings the vehicle to a complete halt for a brief settling period. This pause removes any residual momentum and guarantees that the subsequent reverse-and-turn begins from a known, stationary position, which makes the maneuver far more repeatable.

```
class ParkAlignState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = config.PARK_ALIGN_SPEED
        car.right_pwm = config.PARK_ALIGN_SPEED
        car.command = "PARK_ALIGN"
        if (time.time() - car.state_start_time
                >= config.PARK_ALIGN_DURATION):
            return 'PARK_STOP1'
        return None
```

```
class ParkStop1State(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "PARK_STOP"
        if (time.time() - car.state_start_time
                >= config.PARK_STOP_DURATION):
            return 'PARK_BACK_STEER'
        return None
```

Listing 4.37: Forward alignment and pre-reverse settling states (states.py)

4.6.3 Reverse Turn and Straightening

The core of the maneuver is the ParkBackSteer state, which performs a reverse turn into the slot. The two drive wheels are commanded with deliberately asymmetric PWM values so that the chassis swings into the parking bay as it moves backwards, tracing the curved path required to enter a perpendicular or angled slot from the lane. The asymmetry between the left and right wheel speeds determines the turning radius and is one of the most important calibration parameters of the whole maneuver.

After the timed reverse turn completes, the ParkBackStraight state drives both wheels backward at an equal speed for a short interval, straightening the vehicle and settling it fully inside the slot. The maneuver then advances to its terminal state.

```
class ParkBackSteerState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = config.PARK_BACK_STEER_LEFT
        car.right_pwm = config.PARK_BACK_STEER_RIGHT
        car.command = "PARK_BACK_STEER"
        if (time.time() - car.state_start_time
                >= config.PARK_BACK_STEER_DURATION):
            return 'PARK_BACK_STRAIGHT'
        return None


class ParkBackStraightState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = config.PARK_BACK_STRAIGHT
        car.right_pwm = config.PARK_BACK_STRAIGHT
        car.command = "PARK_BACK_STRAIGHT"
        if (time.time() - car.state_start_time
                >= config.PARK_BACK_STRAIGHT_DURATION):
            return 'PARK_COMPLETE'
        return None
```

Listing 4.38: Reverse-turn and straightening states that enter the slot (states.py)

4.6.4 Parked State and Completion

The final state, ParkComplete, is a terminal holding state. It sets both motor speeds to zero and reports the vehicle as parked, remaining in this state indefinitely so that the car does not drift once it has settled. Because the parking sequence is built from the same State interface as every other behaviour, no special handling is needed in the main loop: the controller simply stops issuing transitions once the terminal state is reached.

```
class ParkCompleteState(State):
    def update(self, car, frame, left_fit, right_fit,
               left_valid, right_valid, mask):
        car.left_pwm = 0
        car.right_pwm = 0
        car.command = "PARKED"
        return None      # terminal state: hold position
```

Listing 4.39: Terminal parked state that holds the vehicle in place (states.py)

Overall, the automatic parking maneuver demonstrates how a relatively complex, multi-stage behaviour can be expressed cleanly within the project's finite-state-machine architecture. By combining a debounced ultrasonic space search with a fixed, well-calibrated sequence of reverse-and-turn motions, the prototype is able to identify a free slot and park itself reliably, extending the system beyond forward driving toward the kind of low-speed self-parking behaviour found in commercial driver-assistance systems.

Chapter 5
System Testing

Chapter 5: System Testing

This chapter describes the testing methodology, test cases, and results for the autonomous car prototype. Each module was tested independently before full system integration testing.

5.1 YOLO Detection Testing

The YOLO model was tested under different lighting conditions and object placements:

- Objects from all four classes (toy cars, Lego persons, stop signs, traffic lights) were placed in the vehicle path.
- Detection was performed at real-time speeds on RPI 5.
- The system correctly identified and classified objects with high confidence scores.

Overall results: Precision 96.8%, Recall 95.6%, mAP@50 of 98.0%, mAP@50-95 of 79.7%.

Figure 5.1 shows a sample of the YOLO detection output on the test objects, with the detected toy car, Lego person, and stop sign bounded and labelled with their confidence scores.

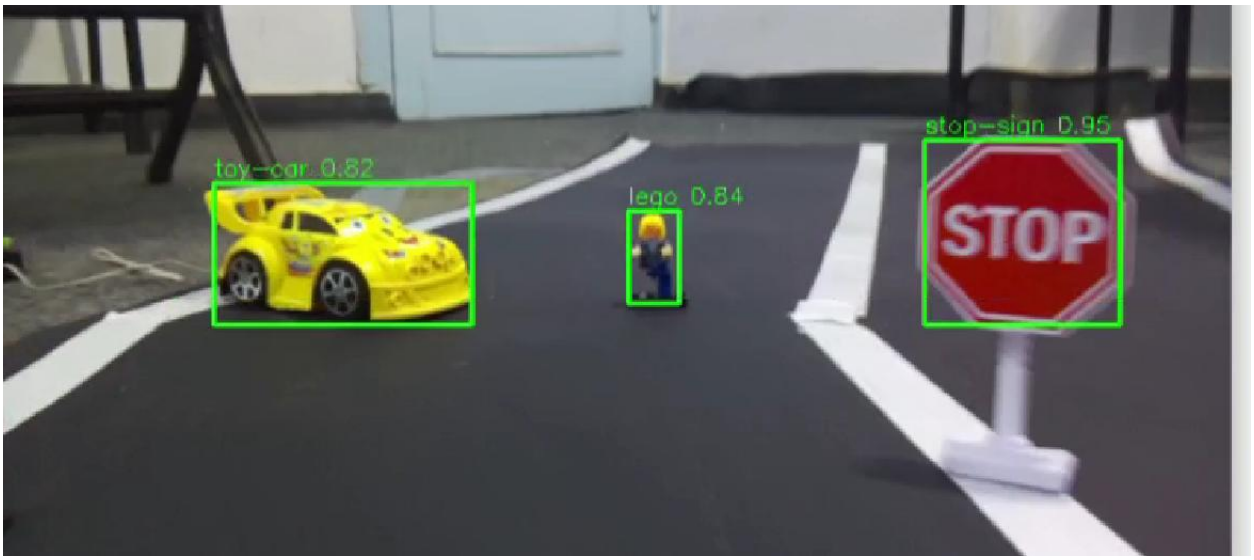


Figure 5.1: YOLO Detection Output on Test Objects

5.2 Lane Detection Testing

Lane detection was tested on a simulated road track:

- The vehicle successfully followed straight road sections.
- The vehicle correctly navigated curved lane sections by adjusting steering proportionally.
- The P-control algorithm maintained lane centering with minimal oscillation.

Figure 5.2 shows the vehicle following the lane on both straight and curved sections of the test track, with the detected lane and steering state overlaid on the camera frame.

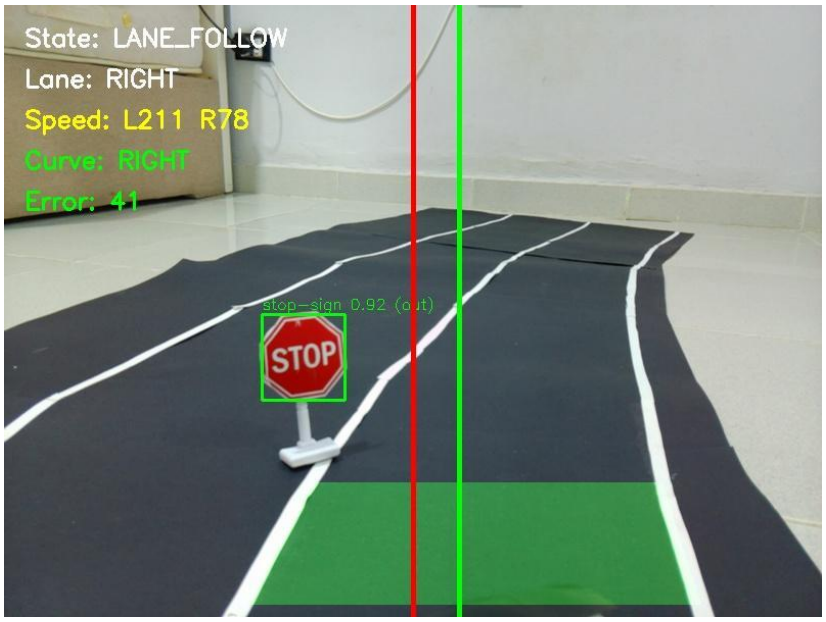


Figure 5.2: Lane Following on Straight and Curved Sections

5.3 Hardware Integration Testing

Full hardware integration was tested by running the complete pipeline:

- Camera frame capture → YOLO detection → Lane detection → Decision logic → Serial command → Arduino → Motor actuation.
- Serial communication between RPI and Arduino was verified with no dropped commands.
- Power supply was confirmed stable under full load conditions.

5.4 Obstacle Avoidance Testing

Obstacle avoidance behavior was tested with objects placed in the vehicle path:

- Stop response: Vehicle stopped when an obstacle was detected directly in front within the danger threshold.
- Slow down response: Vehicle reduced speed for nearby detected objects.
- Steering response: Vehicle adjusted direction when a lane was blocked.

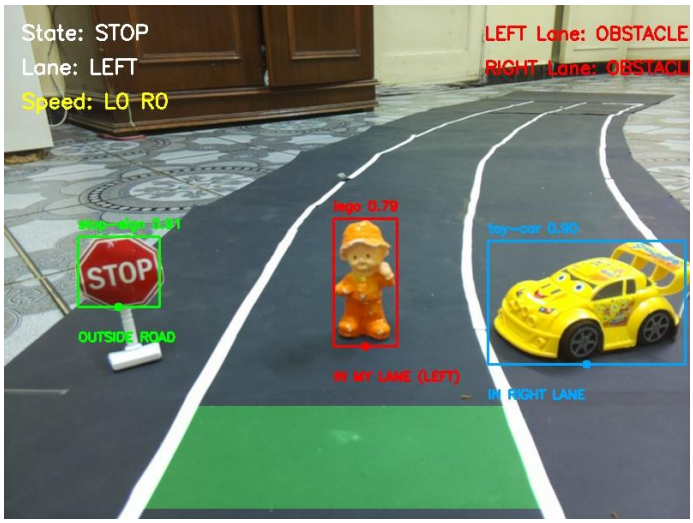


Figure 5.3: Detecting obstacles on two lanes

5.5 Tools and Components

5.5.1 Hardware Tools

Table 5.1: Hardware Tools

Tool	Purpose
RPI 5 (8GB)	Main processing unit for AI inference and decision-making
Arduino Uno R3	Low-level hardware control and motor PWM management
Camera Module (Pi Camera)	Real-time video frame capture
Ultrasonic Sensor (HC-SR04)	Obstacle proximity and distance measurement
L298N Motor Driver	DC motor power control interface
DC Motors + Chassis	Physical vehicle movement and mobility
12V Battery Pack	Main power source for motors
5V / 5A Power Bank	RPI power supply

5.5.2 Software Tools

Table 5.2: Software Tools and Libraries

Tool	Purpose
Python 3	Primary programming language for all software modules
YOLOv8 (Ultralytics)	Object detection model training and inference
OpenCV	Image processing, lane detection, and camera interface
RPI OS	Operating system for RPI 5
Arduino IDE	Programming and uploading code to Arduino Uno
Roboflow	Dataset management, annotation, and augmentation
PySerial	Serial communication between RPI and Arduino
NumPy	Numerical computation and array processing

Chapter 6
Conclusion and Future Work

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project successfully developed a Vision-Based Obstacle Detection and Avoidance System for an Autonomous Car Prototype, integrating deep-learning object detection, classical lane detection, and embedded hardware control into a single real-time pipeline on a low-cost platform. The result demonstrates that meaningful autonomous driving behaviour can be achieved without expensive sensors or high-end computing hardware.

At the core of the system, a custom YOLOv8n model trained from scratch on a merged dataset of 7,311 augmented samples achieved a precision of 96.8%, a recall of 95.6%, an mAP@50 of 98.0%, and an mAP@50-95 of 79.7%, proving accurate enough to drive real-time decisions. These detections were combined with a classical OpenCV lane-detection pipeline that allowed the vehicle to follow straight and curved lanes and to recover when a boundary was briefly lost.

A fifteen-state finite-state machine then coordinated higher-level behaviours including proportional steering, stop-sign and traffic-light response, ultrasonic-fused obstacle avoidance, lane changing, U-turns, and automatic reverse parking, while the Raspberry Pi 5 and Arduino Uno provided real-time processing and dependable motor control linked by a robust serial protocol with a lane-loss fail-safe.

The system's main strengths are its low cost, high detection accuracy, real-time responsiveness, and modular, extensible architecture. Its limitations are equally clear: constrained embedded processing power, sensitivity to complex lighting, a controlled test environment that differs from real roads, and the restricted depth perception of a single camera. Overall, the project met its primary objectives, delivering a functional low-cost prototype that combines accurate detection, stable lane following, dependable control, and a set of advanced manoeuvres, and providing a solid foundation for more capable autonomous driving behaviour.

6.2 Future Work

The following open points represent opportunities for future development:

- Upgrading to stereo cameras or LiDAR for improved depth perception.
- Training the YOLO model on larger real-world datasets for better generalization.
- Adding a web dashboard for real-time performance monitoring.
- Implementing Reinforcement Learning (DQN) for more adaptive decision-making.
- Integrating GPS and IMU sensors for localization and autonomous navigation in larger outdoor environments.
- Optimizing the perception and control pipeline to achieve lower latency and higher frame rates on embedded hardware.
- Expanding the traffic-sign recognition module to support additional road signs such as speed limits, pedestrian crossings, yield signs, and no-entry signs.
- Implementing Behavioral Cloning to learn steering and speed commands directly from human driving demonstrations, reducing the need for manually designed control rules and enabling more natural driving behavior.

References

[1] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao and K. Zieba, "End to End Learning for Self-Driving Cars," arXiv preprint, arXiv:1604.07316, Apr. 2016.

[2] C. Chen, A. Seff, A. Kornhauser, and J. Xiao, "DeepDriving: Learning Affordance for Direct Perception in Autonomous Driving," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2722-2730, 2015.

[3] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, and S. Guadarrama, "Speed/Accuracy Trade-offs for Modern Convolutional Object Detectors," in Computer Vision and Pattern Recognition (CVPR), 2017.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[5] Ultralytics, "YOLOv8: Ultralytics YOLOv8 Docs," https://docs.ultralytics.com, Last Retrieved 2025.

[6] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks," in European Conference on Computer Vision (ECCV), 2016.

[7] K. Lee and H. Peng, "Evaluation of Automotive Forward Collision Warning and Collision Avoidance Algorithms," Vehicle System Dynamics, vol. 43(10), pp. 735-751, Feb. 2007.

[8] C. Stiller, J. Hipp, C. Rossig, and A. Ewald, "Multisensor Obstacle Detection and Tracking," Image and Vision Computing, vol. 18(5), pp. 389-396, 2000.